



Mohamed Sathak
A J College of Engineering
An Autonomous Institution
SATHAK - CHENNAI



Approved by AICTE and UGC (12B & 2f)
Affiliated to Anna University
Accredited by NAAC Grade A & NBA
An ISO: 21001-2018 Certified Institution

LAB RECORD

Name of the Student:

Register Number :

Year/Semester :

Department :

Academic Year :



Mohamed Sathak
A J College of Engineering
An Autonomous Institution
SATHAK - CHENNAI



Approved by AICTE and UGC (12B & 2f)
Affiliated to Anna University
Accredited by NAAC Grade A & NBA
An ISO:21001-2018 Certified Institution

Vision

- To be an eminent institute for higher education and research through innovative teaching-learning and sustainable practices to meet the industrial and societal needs.

Mission

- To impart quality technical education by transforming students into professionally competent technocrats.
- To nurture the students in all-round sustainable development, for career and self employability.
- To inculcate critical thinking, professional ethics with civic responsibilities by instilling values.

Quality Policy

- MSAJCE committed to create quality professionals through innovative teaching, research, industrial relations and nurturing human values & ethics to meet the industrial and societal needs on continual improvement basis.

MECHANICAL ENGINEERING

REGULATION 2021

IV YEAR/VII SEMESTER

SUB CODE: ME3781

MECHATRONICS AND IOT LABORATORY

LAB MANUAL



Mohamed Sathak
A J College of Engineering
An Autonomous Institution
SATHAK - CHENNAI



Approved by AICTE and UGC (12B & 2f)
Affiliated to Anna University
Accredited by NAAC Grade A & NBA
An ISO:21001-2018 Certified Institution

BONAFIDE CERTIFICATE

This is to certify that Mr./Ms..... of
..... year Semester B.E. / B. Tech (.....
.....) is a bonafide student bearing the
Reg. No. and record of work done in the
..... lab during
the academic year 20 - 20

Faculty in-charge

Head of the Department

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

Index

S. No	TOPIC	PAGE NO	
1.	Syllabus		
2.	List of Experiments		MARK S
1	Measurement of Linear/Angular of Position, Direction and Speed using Transducers.		
2	Measurement of Pressure, Temperature and Force using Transducers.		
3	Speed and Direction control of DC Servomotor, AC Servomotor and Induction motors.		
4	Addition, Subtraction and Multiplication Programming in 8051.		
5	Programming and Interfacing of Stepper motor and DC motor using 8051/PLC.		
6	Programming and Interfacing of Traffic Light Interface using 8051.		
7	Sequencing of Hydraulic and Pneumatic circuits.		
8	Sequencing of Hydraulic, Pneumatic and Electro-pneumatic circuits using Software.		
9	Electro-pneumatic/hydraulic control using PLC.		
10	Vision based image acquisition and processing technique for inspection and classification.		
11	Familiarization with concept of IoT and its open-source microcontroller/SBC.		
12	Write a program to turn ON/OFF motor using microcontroller/SBC through internet.		
13	Write a program to interface sensors to display the data on the screen through internet.		
14	Interface the sensors with microcontroller/SBC and write a program to turn ON/OFF Solenoid valve through internet when sensor data is detected.		
15	To interface sensor with microcontroller/SBC and write a program to turn ON/OFF Linear/Rotary Actuator through IoT when sensor data is detected.		
16	To interface Bluetooth/Wifi with microcontroller/SBC and write a program to send sensor data to smart phone using Bluetooth/wifi.		

LIST OF EXPERIMENTS

Mechatronics

1. Measurement of Linear/Angular of Position, Direction and Speed using Transducers.
2. Measurement of Pressure, Temperature and Force using Transducers.
3. Speed and Direction control of DC Servomotor, AC Servomotor and Induction motors.
4. Addition, Subtraction and Multiplication Programming in 8051.
5. Programming and Interfacing of Stepper motor and DC motor using 8051/PLC.
6. Programming and Interfacing of Traffic Light Interface using 8051.
7. Sequencing of Hydraulic and Pneumatic circuits.
8. Sequencing of Hydraulic, Pneumatic and Electro-pneumatic circuits using Software.
9. Electro-pneumatic/hydraulic control using PLC.
10. Vision based image acquisition and processing technique for inspection and classification.

Internet of Things

11. Familiarization with concept of IoT and its open-source microcontroller/SBC.
12. Write a program to turn ON/OFF motor using microcontroller/SBC through internet.
13. Write a program to interface sensors to display the data on the screen through internet.
14. Interface the sensors with microcontroller/SBC and write a program to turn ON/OFF Solenoid valve through internet when sensor data is detected.
15. To interface sensor with microcontroller/SBC and write a program to turn ON/OFF Linear/Rotary Actuator through IoT when sensor data is detected.
16. To interface Bluetooth/Wifi with microcontroller/SBC and write a program to send sensor data to smart phone using Bluetooth/wifi.

Experiment: 1	Measurement of Linear/Angular of Position, Direction and Speed using Transducers.
Date:	

AIM:

To measure linear position, angular position, speed, and direction of a mechanical system using appropriate transducers.

Apparatus Required:

S. No	Apparatus	Specification/Range	Quantity
1	Linear Position Sensor (LVDT / Linear Potentiometer)	0–50 mm	1
2	Rotary Encoder (Angular Position)	0–360°	1
3	Tachogenerator / Speed Transducer	0–3000 rpm	1
4	Direction Sensor / Limit Switch	—	2
5	Power Supply	0–24 V DC	1
6	Multimeter / Data Acquisition System	—	1
7	Connecting Wires	—	As needed

PROCEDURE:

1. Linear Position Measurement

- ✚ Connect linear potentiometer or LVDT to the moving element.
- ✚ Move the element to different positions and record output voltage.
- ✚ Plot displacement vs output voltage.

2. Angular Position Measurement

- ✚ Mount rotary encoder on the rotating shaft.
- ✚ Rotate shaft through known angles and record pulses or voltage.
- ✚ Plot angular position vs output.

3. Speed Measurement

- ✚ Connect tachogenerator to motor shaft.
- ✚ Vary motor speed using controller or VFD.
- ✚ Measure output voltage/current corresponding to speed.
- ✚ Plot speed vs voltage.

4. Direction Measurement

- ✚ Connect limit switches or direction sensors.
- ✚ Move shaft or element forward/reverse.
- ✚ Observe sensor signals for CW/CCW or forward/backward movement.

TABULATION

1. Linear Position Sensor

Displacement (mm)	Sensor Voltage (V)
0	0.2
10	1.1
20	2.1
30	3.0
40	3.9
50	5.0

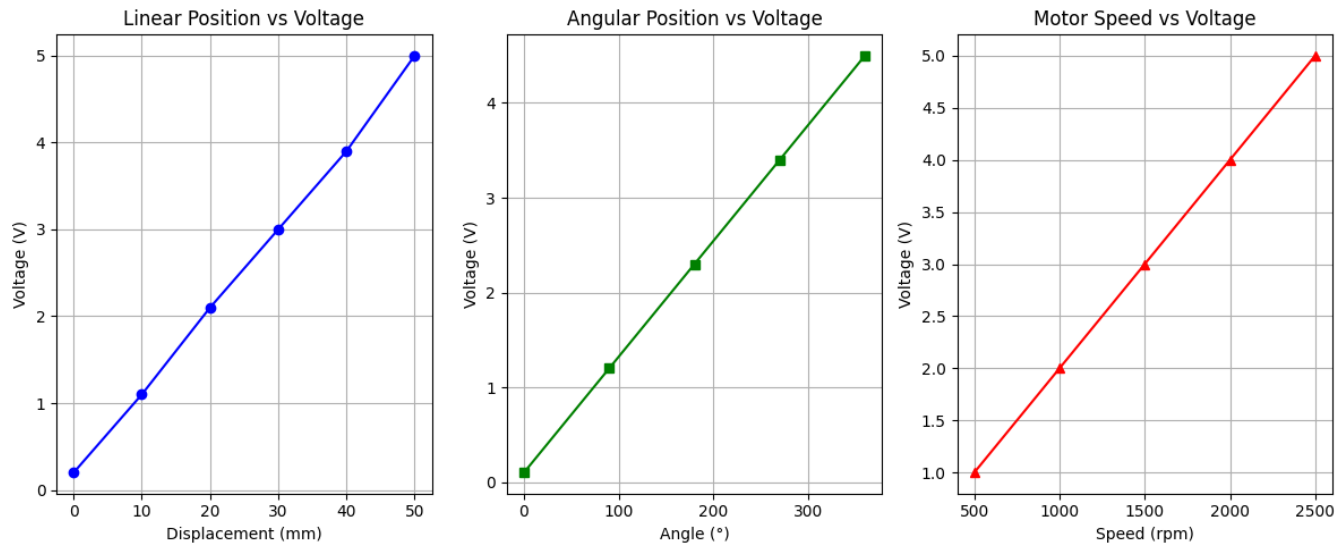
2. Angular Position (Rotary Encoder)

Angle (°)	Encoder Pulses	Output Voltage (V)
0	0	0.1
90	250	1.2
180	500	2.3
270	750	3.4
360	1000	4.5

3. Speed Measurement (Tachogenerator)

Motor Speed (rpm)	Sensor Voltage (V)	Direction
500	1.0	CW
1000	2.0	CW
1500	3.0	CW
2000	4.0	CCW
2500	5.0	CCW

Model Graph:



Result:

Hence the experiment to measure linear position, angular position, speed, and direction using transducers was successfully conducted.

- Linear displacement was measured accurately using LVDT/potentiometer with output voltage proportional to position.
- Angular position was measured with a rotary encoder, showing linear variation of voltage/pulses with angle.
- Motor speed and direction were measured using a tachogenerator and direction sensors, with output voltage proportional to speed and correct direction detected.

Experiment: 2	Measurement of Pressure, Temperature and Force using Transducers.
Date:	

Aim: To study temperature transducers RTD by using digital temperature indicator.

Apparatus Required:

Apparatus	Specification / Range	Quantity
Strain-gauge Pressure Transducer	0–10 bar or 0–20 bar	1
Pressure Pump (air/hydraulic)	0–20 bar	1
Reference Pressure Gauge	0–20 bar, accuracy $\pm 1\%$	1
DC Power Supply	0–30 V, 0–2 A	1
Signal Conditioner / Amplifier	Gain 0–1000	1
Digital Indicator / DAQ System	Compatible with transducer	1
Connecting Cables	Standard shielded cables	As per required

1. Temperature Measurement

Apparatus	Specification / Range	Quantity
Thermocouple (Type K or J)	–50°C to 1200°C	1
RTD (Pt100)	–50°C to 400°C	1
Temperature Transducer	Ambient to 300°C	1
Thermocouple/RTD Reader	mV/ Ω measurement capability	1
Constant Current Source (for RTD)	0–5 mA	1
Wheatstone Bridge / Signal Conditioner	For RTD calibration	1
Digital Thermometer (Reference)	–50°C to 300°C	1
Connecting Wires	Heat-resistant	As per required

2. Force Measurement

Apparatus	Specification / Range	Quantity
Strain-gauge Load Cell	0–500 N or 0–1000 N	1
Set of Standard Weights	1 N to 500 N (or equivalent mass)	1 set
Load Frame / Support Stand	Suitable for load cell	1
Signal Conditioner / Strain Gauge Amplifier	Gain 0–1000	1
DC Power Supply	0–15 V	1
Digital Indicator / DAQ System	Compatible with load cell	1
Connecting Cables	Shielded	As required

Procedure:

1. Switch on the trainer kit.
2. The display indicates the room temperature if not by varying gain pot set the room temperature.
3. Note down the temperature and voltage at the test point T3.
4. Dip the RTD sensor into the water bath.
5. Switch on the water bath.
6. Dip the thermometer into the water bath.
7. Note down the actual temperature in the thermometer, display temperature and voltage at T5 for every 10 degrees.
8. Draw graph actual temperature vs display temperature.

RTD :

Equation For Calculating R_t

$$R_t = R_0 (1 + \alpha * \Delta T)$$

Where,

R_t is resistance at measurement temperature

R_0 is resistance at reference (0°C) temperature

α is temperature co-efficient of material

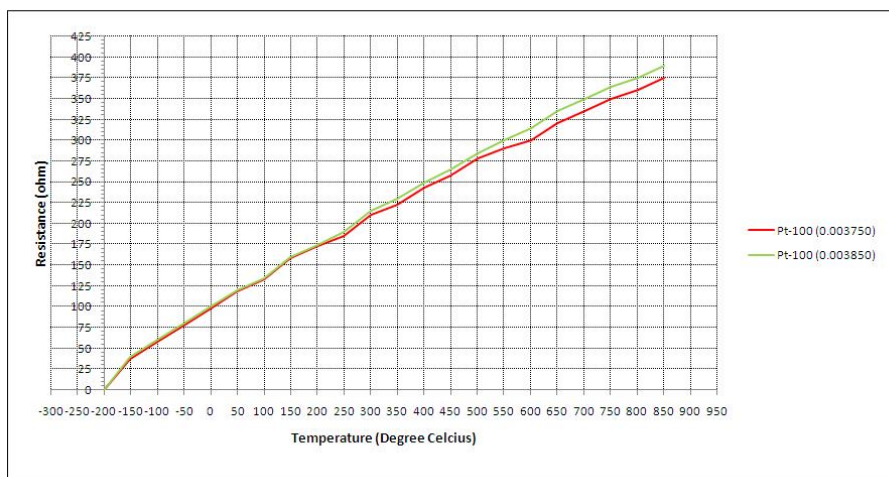
ΔT is the difference between measurement and reference temperature

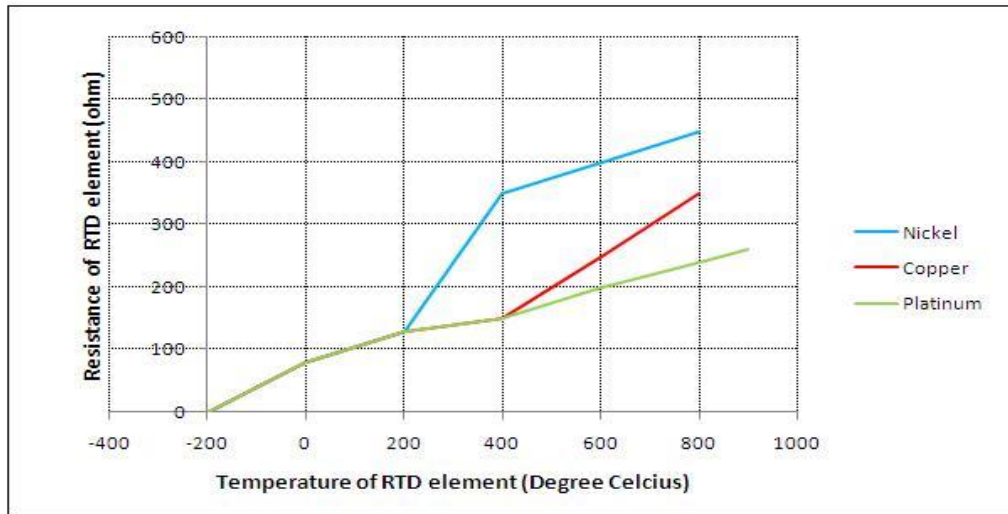


The requirements of a conductor to be used in RTD are:

1. The change in resistance of a material per unit change in temperature should be as large as possible.
2. The material should have a high value of resistivity so that the minimum volume of material is used for the construction of RTD.
3. The resistance of material should have a continuous and stable relationship with temperature.
4. Copper is used occasionally as an RTD element.
5. It is to resistivity forces the element to be larger than the platinum element but it is low linearly and low costs making it is an economical alternative.
6. Its upper limit of temperature 120 0 c.
7. The most common RTD are made of either platinum, nickel or nickel alloys. The economical nickel wires are used over a limited temperature range, they are quiet long linear and tend to draft with time for measurement integrity platinum is the obvious choice the single most common value is 100
8. Ω at 0 0 C with resistance temperature co-efficient 0.00385/ 0 C the more chemically pure platinum wire has resistance temperature co-efficient 0.00392/ 0 C

Model Graph:



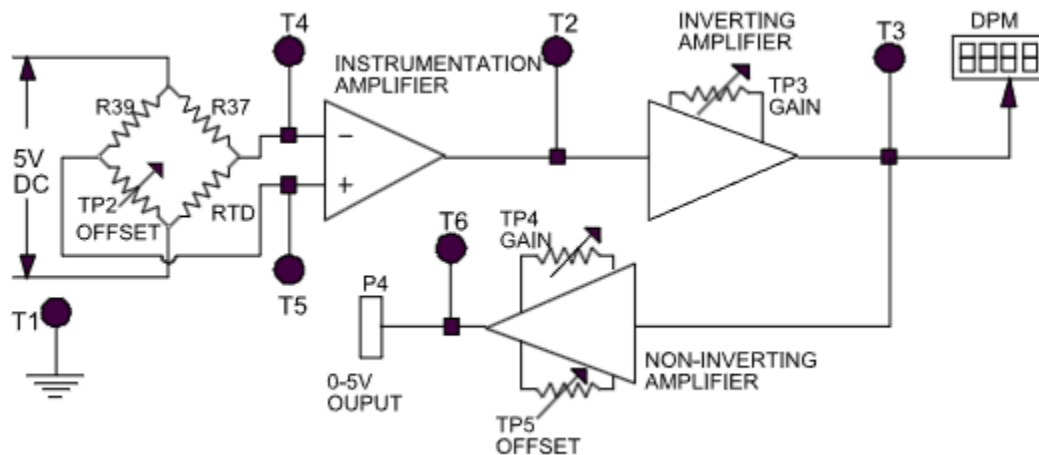


Electrical Resistance-Temperature Curves

Type: PT 100

Resistance value: $100\ \Omega$ at $0\ 0\ ^\circ\text{C}$

BLOCK DIAGRAM:



Tabulation:

SNO	Actual Temperature	Display Temperature	Voltage at o/p
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Result:

Hence, the Temperature of transducer RTD were calibrated using digital thermometer and output has been tabulated.

Experiment: 3	Speed and Direction control of DC Servomotor, AC Servomotor and Induction motors.
Date: 19/11/2025	

Aim:

To study and perform **speed and direction control** of

1. **DC Servomotor**
2. **AC Servomotor**
3. **Induction Motor**

using appropriate control circuits and to observe the behaviour of each motor under varying input and load conditions.

2. APPARATUS REQUIRED**(A) DC Servomotor**

S. No	Apparatus	Range/Specification	Quantity
1	DC Servomotor	12V / 24V	1
2	DC Servo Driver or H-Bridge (L298/L293D)	12–24V	1
3	Variable DC Power Supply	0–30V, 2A	1
4	Potentiometer (Speed control)	10k Ω	1
5	Microcontroller/servo tester (optional)	PWM output	1
6	Tachometer	0–5000 rpm	1
7	Connecting wires	—	As required

(B) AC Servomotor

S. No	Apparatus	Range/Specification	Quantity
1	AC Servomotor	1-phase, 230V	1
2	Servo amplifier/AC driver	230V input	1
3	Autotransformer (Variac)	0–260V	1
4	Direction control switch	1-phase reversing	1
5	Tachometer	0–5000 rpm	1
6	Connecting wires	—	As required

(C) Induction Motor

S. No	Apparatus	Range/Specification	Quantity
1	3-phase Induction Motor	0.5–3 HP	1
2	DOL Starter / VFD	415V	1
3	3-phase auto transformer or VFD	0–415V	1
4	Direction control switch / VFD terminal	—	1
5	Tachometer	0–6000 rpm	1
6	Ammeter, voltmeter	Suitable range	1 each
7	Connecting cables	—	As required

3. PROCEDURE

(A) DC Servomotor Control

Speed Control

1. Connect the DC servomotor to the driver (H-Bridge or servo driver).
2. Connect the power supply (12/24V) to the motor driver.
3. Connect a potentiometer or PWM source to the control input.
4. Gradually increase PWM duty cycle or voltage.
5. Measure motor speed using a tachometer.
6. Record voltage vs speed.

Direction Control

1. Use the H-Bridge input pins:
 - IN1 = HIGH, IN2 = LOW → Motor rotates clockwise
 - IN1 = LOW, IN2 = HIGH → Motor rotates counter-clockwise
2. Reverse polarity and observe direction change.

(B) AC Servomotor Control

Speed Control

1. Connect the AC servomotor to the servo amplifier.
2. Supply power to the servo amplifier.
3. Adjust the input control (analog/pulse).
4. Gradually change input to vary speed.
5. Note speed using a tachometer.

Direction Control

1. Use the direction input on the servo amplifier.
2. Set DIR = HIGH → Clockwise
3. Set DIR = LOW → Counter-clockwise
4. Alternatively, use a reversing switch (swap phase).

(C) Induction Motor Control

Speed Control (Using VFD)

1. Connect induction motor to VFD output terminals (U-V-W).
2. Connect VFD to AC supply.
3. Power ON the VFD.
4. Set frequency from 10 Hz → 50 Hz gradually.
5. Observe change in motor speed.
6. Record frequency & speed.

Direction Control

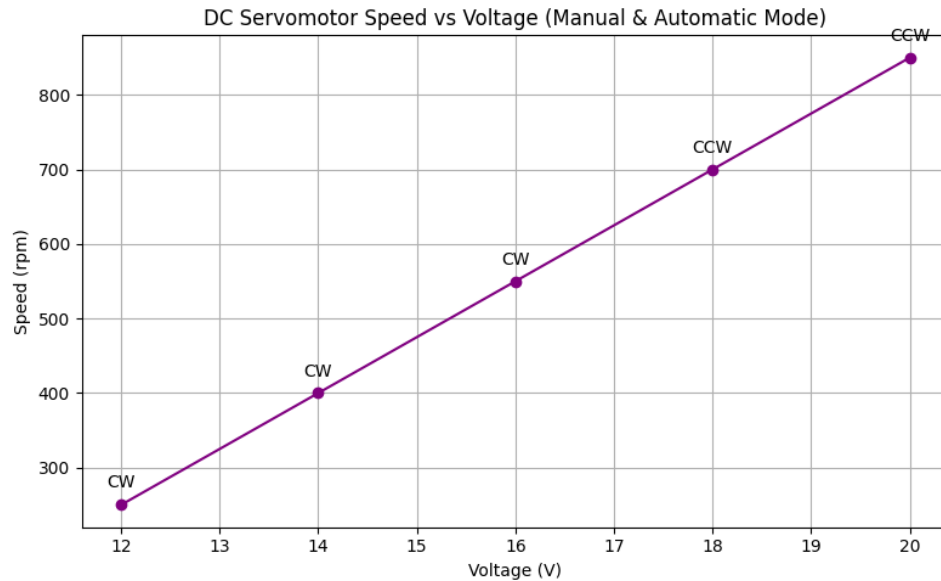
1. On VFD: press **REV** (reverse run).
2. Or manually interchange any two motor terminals (R-Y, Y-B, or B-R).
3. Observe reversal of rotation.

4. TABULATION FORMAT

(A) DC Servomotor (Manual and Automatic Mode)

Speed Control Table

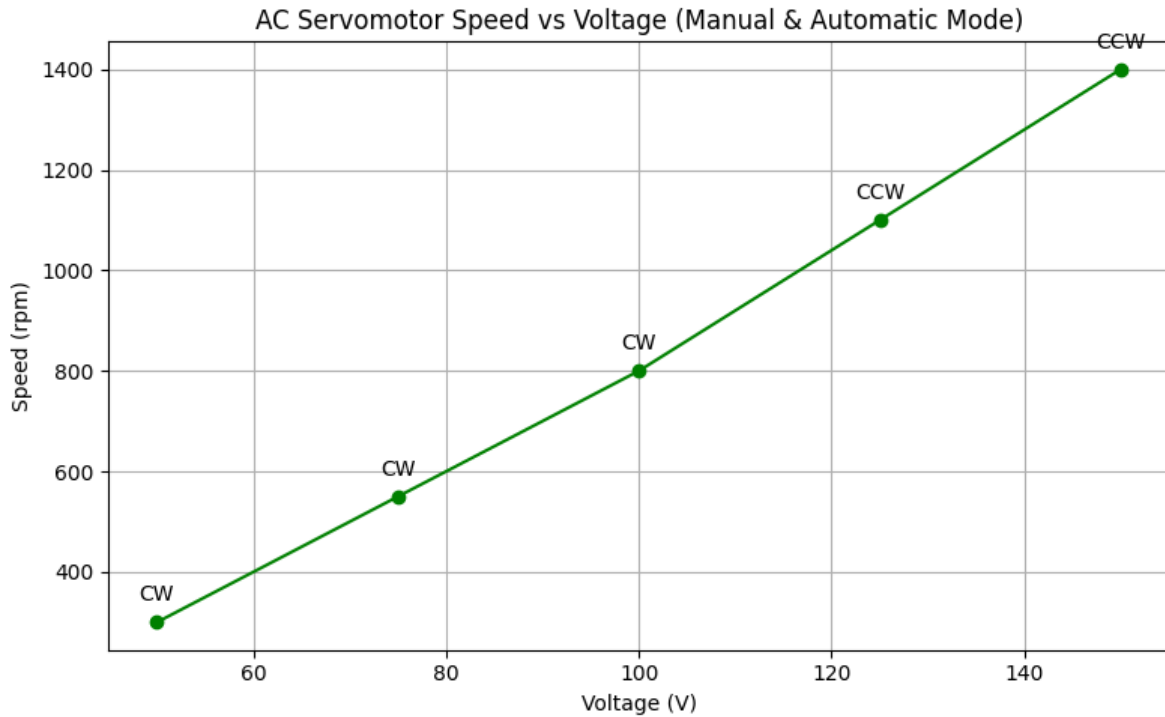
Voltage (V)	PWM Duty (%)	Speed (rpm)	Direction
12	20	250	CW
14	35	400	CW
16	50	550	CW
18	60	700	CCW
20	80	850	CCW



(B) AC Servomotor (Manuel and Automatic Mode) Speed Control Table

Voltage (V)	PWM Duty (%)	Speed (rpm)	Direction
50 V	20%	300 rpm	CW
75 V	35%	550 rpm	CW
100 V	50%	800 rpm	CW
125 V	65%	1100 rpm	CCW
150 V	80%	1400 rpm	CCW

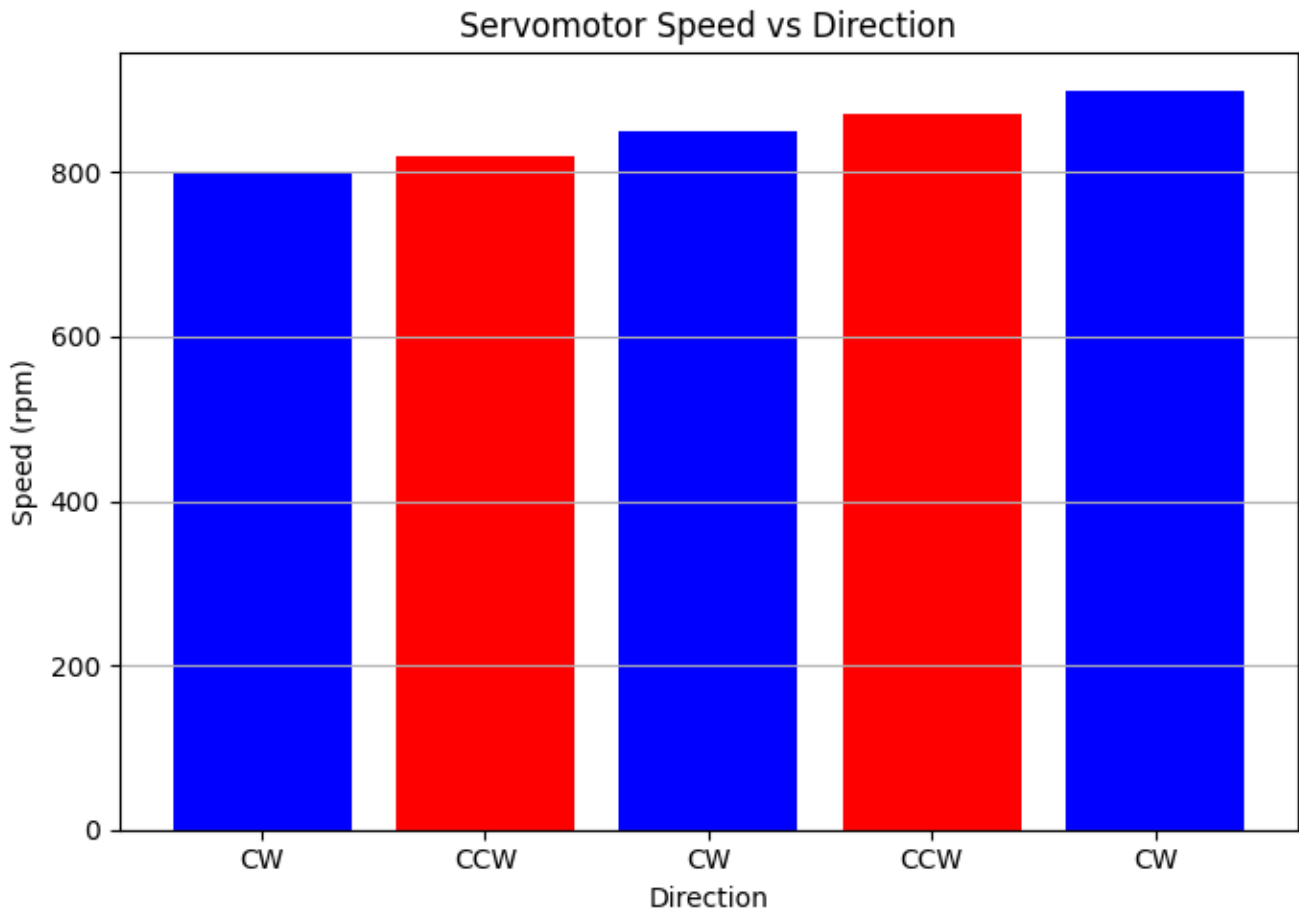
Model Graph:



Direction Control Table

Input Logic (IN1, IN2)	Direction Observed	Speed (rpm)
HIGH, LOW	CW	800
LOW, HIGH	CCW	820
HIGH, LOW	CW	850
LOW, HIGH	CCW	870
HIGH, LOW	CW	900

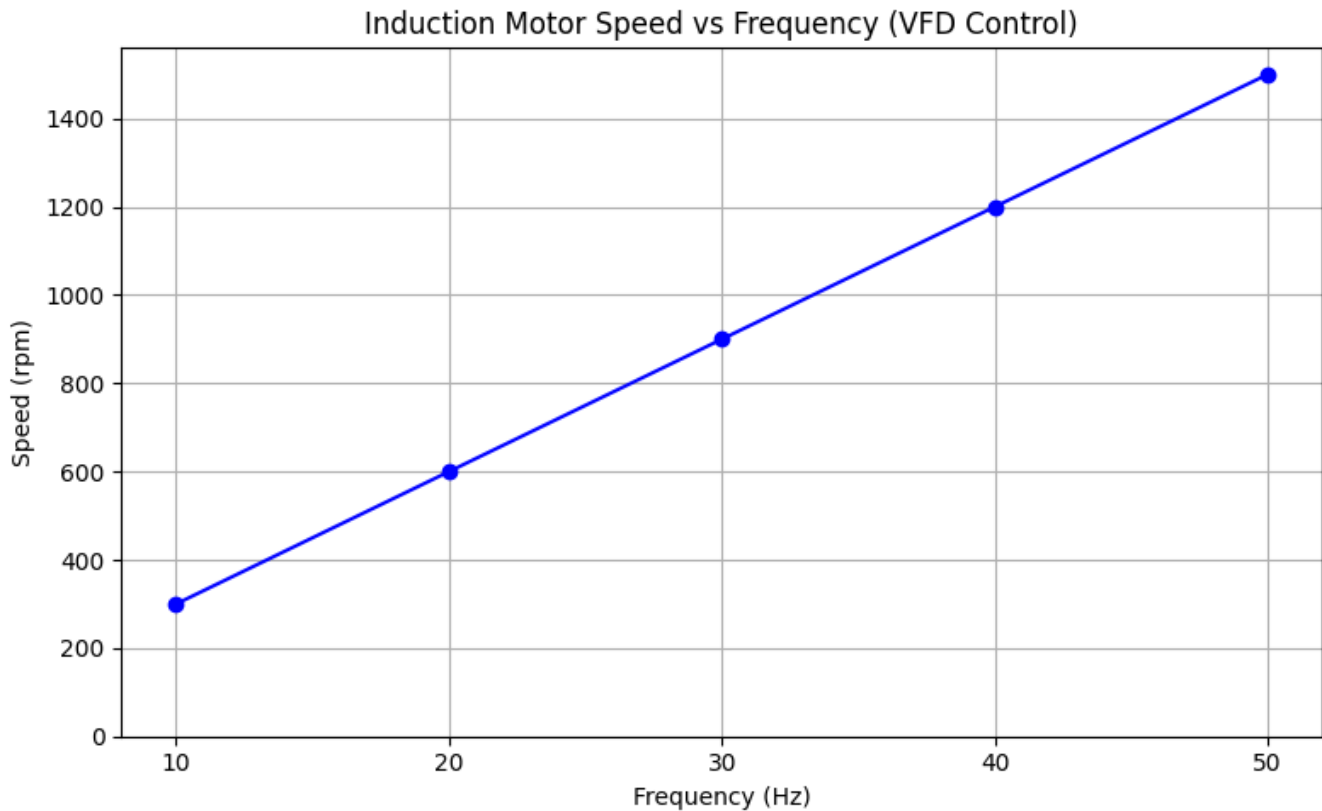
Model Graph:



C. Induction Motor (VFD Control)

Frequency (Hz)	Voltage (V)	Speed (rpm)	Current (A)	Direction
10 Hz	80 V	300 rpm	0.9 A	Forward
20 Hz	160 V	600 rpm	1.2 A	Forward
30 Hz	240 V	900 rpm	1.6 A	Forward
40 Hz	320 V	1200 rpm	1.9 A	Forward
50 Hz	400 V	1500 rpm	2.2 A	Reverse

Model Graph: Induction Motor (VFD Control)



Result:

Hence all three motors DC servomotor, AC servomotor, and induction motor were successfully controlled in **speed and direction** using manual and automatic methods.

- **DC Servo:** Easy PWM control, precise direction.
- **AC Servo:** Fast response, high accuracy, ideal for automation.
- **Induction Motor (VFD):** Smooth speed variation, reliable for industrial application.

Experiment: 4	Addition, Subtraction and Multiplication Programming in 8051.
Date:	

Aim:

To write and execute Assembly language programs for the 8051 microcontroller to perform:

1. Addition
2. Subtraction
3. Multiplication

The results will be stored in the accumulator or specified memory locations.

Apparatus Required:

SNO	Equipment name	Quantity
1	8051 Microcontroller Trainer Kit or 8051 Development Board	1
2	Keil μVision / MASM / MC51 Compiler	1
3	USB/Serial Programmer (if loading onto hardware)	1
4	Power Supply	1
5	Connecting Cables	1
6	Computer System with 8051 IDE installed	1

Procedure:**1. Power ON the Trainer Kit**

1. Connect the **power chord** to the microprocessor kit.
2. Switch **ON** the kit.
3. The display will show the **monitor program** (e.g., “READY”, “0000”, or similar).

2. Enter Program Mode

1. Press the **ADDRESS (ADDR)** key on the keyboard.
2. Enter the starting memory address where you want to store your program (commonly **8000H**, **0000H**, or **2000H** depending on your kit).
3. Press **ENTER**.

3. Enter the Program Into Memory

1. Type the **hex opcode** for the first instruction.
2. Press **ENTER**.
3. The address will automatically increment to the next location.
4. Continue entering **all remaining opcodes** in sequence.

4. Save / Exit Program Entry

After entering all program bytes:

- Press **RESET**
or
- Press **ESC** (some kits use this)

5. Execute (Run) the Program

1. Press **GO / RUN** key.
2. Enter the **starting address** of the program (e.g., 2000H).
3. Press **ENTER** to execute.

The microprocessor executes the stored program.

6. View the Results

Use the **REGISTER** or **DISPLAY** key to check:

For Addition

- Result is in **A (Accumulator)**
- **CY flag** indicates carry

For Subtraction

- Result is in **A**
- **CY flag** indicates borrow

For Multiplication

- **A** = Lower byte of result

- **B** = Upper byte of result

Check values on the 7-segment or LCD display of the kit.

7. Verify the Output

1. Compare the displayed output with your expected manual calculation.
2. If wrong:
 - Re-check opcodes.
 - Re-enter incorrect memory values.
 - Run again.

8. End the Experiment

1. Press **RESET** to stop the program.
2. Switch **OFF** the kit.
3. Disconnect the power chord safely.
4. Record the results in your observation table.

CODE:ADDER

ADDR → 2000 → ENTER

2000 : 74 (MOV A,#data)

ENTER

2001 : 25 (data)

ENTER

2002 : A9 (MOV R1,#data)

ENTER

2003 : 37 (data)

ENTER

2004 : 29 (ADD A,R1)

ENTER

SUBTRACTOR: Assembly Code

ORG 0000H

MOV A, #45H ; first number

MOV R1, #28H ; second number

CLR C ; clear borrow before subtraction

SUBB A, R1 ; A = A – R1

SJMP \$; stop program (infinite loop)

END

Hex Machine Code (for Trainer Kit Entry)

Hex Machine Code (for Trainer Kit Entry):

Address	Hex Code	Instruction
2000	74	MOV A,#45H
2001	45	data (45H)
2002	79	MOV R1,#28H
2003	28	data (28H)
2004	C3	CLR C
2005	99	SUBB A,R1
2006	80	SJMP 00H
2007	00	displacement

MULTIPLICATION:**Assembly Code**

ORG 0000H

MOV A, #12H ; first number

MOV B, #04H ; second number

MUL AB ; $A \times B$

SJMP \$; infinite loop

END

Hex Machine Code (for Trainer Kit Entry)

Address	Hex Code	Instruction
2000	74	MOV A, #12H
2001	12	data (12H)
2002	F4	MOV B, #04H
2003	04	data (04H)
2004	A4	MUL AB
2005	80	SJMP 00H
2006	00	displacement

✓ Enter these hex codes at 2000H

✓ Run from 2000H

Result:

Hence, the Assembly language programs for Addition, Subtraction, and Multiplication using the 8051 microcontroller were successfully written, executed, and the results were obtained and stored in the specified memory locations.

Experiment: 5	Programming and Interfacing of Stepper motor and DC motor using 8051/PLC.
Date:	

AIM:

1. To write and execute a program to interface and control a Stepper Motor using 8051/PLC.
2. To write and execute a program to interface and control a DC Motor using 8051/PLC.

APPARATUS REQUIRED:

SNO	Equipment Name	Quantity
1	8051 Microcontroller Trainer Kit / PLC Trainer	1
2	Stepper Motor	1
3	DC Motor	1
4	ULN2003 / ULN2803 Driver IC (for Stepper Motor)	1
5	Motor Driver (L293D / Relay Module for DC Motor)	1
6	Connecting Wires	1
7	Power Supply (5V/12V)	1
8	Keyboard (for kit)	1

Stepper Motor

A stepper motor rotates in discrete steps. 8051 sends a sequence of digital pulses to the driver IC (ULN2003) which energizes the coils in order, causing rotation.

Common sequence for **Unipolar Stepper Motor (4-step):**

A	B	C	D
1	0	0	0
1	1	0	0
0	1	1	0
0	0	1	1

DC Motor

A DC motor rotates when voltage is applied.

8051 cannot directly drive the motor, so a driver (L293D/relay) is interfaced.

By controlling the driver input, we can rotate the motor clockwise or stop it.

PROCEDURE (Using 8051 Microprocessor Kit + Keyboard + Motors)**STEPPER MOTOR**

1. Connect stepper motor coil wires to ULN2003 driver output.
2. Connect driver input pins to **Port 2** of 8051 (P2.0–P2.3).
3. Connect motor supply (5V/12V depending on motor).
4. Switch ON the 8051-kit using power chord.
5. Enter the machine code for stepper motor program into memory using keyboard.
6. Run the program from the starting address.
7. Observe the stepper motor rotating step-by-step.
8. Stop the program and switch OFF the kit.

DC MOTOR

1. Connect the DC motor to L293D driver output.
2. Connect L293D control pins to **P1.0 and P1.1** of 8051.
3. Power the driver with 12V supply.
4. Turn ON the trainer kit.
5. Enter the DC motor control program into memory.
6. Run the program and observe clockwise rotation.
7. Stop and record observations.

8051 ASSEMBLY PROGRAMS

1. STEPPER MOTOR PROGRAM (4-step Sequence)

```
ORG 0000H

MOV P2, #0A0H ; 1010 0000 (step 1)
ACALL DELAY

MOV P2, #090H ; 1001 0000 (step 2)
ACALL DELAY

MOV P2, #050H ; 0101 0000 (step 3)
ACALL DELAY

MOV P2, #060H ; 0110 0000 (step 4)
ACALL DELAY

SJMP 0000H ; repeat forever

DELAY:
MOV R7, #255
DE1: MOV R6, #255
DE2: DJNZ R6, DE2
    DJNZ R7, DE1
    RET
END
```

2. DC MOTOR PROGRAM (Clockwise Rotation)

Using L293D → IN1=P1.0, IN2=P1.1

Assembly Code:

```
ORG 0000H

SETB P1.0 ; IN1 = 1
CLR P1.1 ; IN2 = 0 → Motor rotates clockwise

AGAIN: SJMP AGAIN

END
```

REVERSE DIRECTION:

CLR P1.0 ; IN1 = 0

SETB P1.1 ; IN2 = 1 → Motor rotates anticlockwise

RESULT:

Hence the stepper motor interfaced with 8051/PLC rotates in proper stepping sequence, Also the DC motor rotates in the desired direction using driver circuits and 8051/PLC control signals.

Experiment: 6	Programming and Interfacing of Stepper motor and DC motor using 8051/PLC.
Date:	

AIM:

To write, execute, and interface a Traffic Light Control System using the 8051 microcontroller, where RED, YELLOW, and GREEN LEDs glow in a fixed timing sequence.

Apparatus Required:

SNO	Equipment Name	Quantity
1	8051 Microcontroller Trainer Kit	1
2	Traffic Light Interface Board (Red, Yellow, Green LEDs)	1
3	Connecting Wires	1
4	Delay Timer / Resistors (on board)	1
5	Keyboard (for entering program)	1
6	Power chord (to power the 8051 kit)	1
7	8051 Microcontroller Trainer Kit	1
8	Traffic Light Interface Board (Red, Yellow, Green LEDs)	1

THEORY

A traffic light controller operates three LEDs:

- **RED** – Stop
- **YELLOW (AMBER)** – Wait
- **GREEN** – Go

Using the 8051, we set output pins (usually **Port 1**) to HIGH or LOW to turn LEDs ON/OFF through a driver IC (ULN2003/L293D).

Typical sequence:

Step	Light ON	Duration
1	RED	5 sec
2	YELLOW	2 sec
3	GREEN	5 sec

PROCEDURE (Using 8051 Trainer Kit + Keyboard)

1. Hardware Setup

- Connect the Traffic Light Interface board to **Port 1** of 8051.
- Red LED → P1.0
- Yellow LED → P1.1
- Green LED → P1.2
- Ensure all ground and Vcc connections are proper.

2. Powering the Kit

- Connect the **power chord** and switch ON the 8051 trainer kit.

3. Program Entry

- Enter **program mode** on the kit using the keyboard (ADDR key).
- Select a starting address (e.g., **2000H**).
- Enter the machine codes one by one.

4. Execution

- Press **RUN/GO**.
- Enter starting address (2000H).
- LEDs will glow in the traffic sequence (RED → YELLOW → GREEN).

5. Observation

- Note the ON/OFF pattern of LEDs.
- Verify that delays match expected timings.

6. Termination

- Press RESET and power OFF the kit.

8051 ASSEMBLY PROGRAM (Traffic Light Controller)

Assume:

- P1.0 → RED
- P1.1 → YELLOW
- P1.2 → GREEN

ORG 0000H**MAIN:****; ----- RED ON -----**

SETB P1.0 ; Red ON

CLR P1.1 ; Yellow OFF

CLR P1.2 ; Green OFF

ACALL DELAY5S ; 5 second delay

; ----- YELLOW ON -----

CLR P1.0 ; Red OFF

SETB P1.1 ; Yellow ON

CLR P1.2 ; Green OFF

ACALL DELAY2S ; 2 second delay

; ----- GREEN ON -----

CLR P1.0

CLR P1.1

SETB P1.2 ; Green ON

ACALL DELAY5S ; 5 second delay

SJMP MAIN ; repeat forever

; ---- DELAY SUBROUTINES ----**DELAY5S:**

MOV R7,#5 ; 5 x 1-second delay

D5: ACALL DELAY1S

DJNZ R7,D5

RET

DELAY2S:

MOV R7,#2 ; 2 x 1-second delay

D2: ACALL DELAY1S

DJNZ R7,D2

RET

; ---- 1 SECOND DELAY ----

DELAY1S:

MOV R6,#200

D1: MOV R5,#250

D2: DJNZ R5,D2

DJNZ R6,D1

RET

END

HEX MACHINE CODE TABLE (8051 KIT):

HEX MACHINE CODE version of the Traffic Light Program for your **8051 microprocessor kit**.

MAIN PROGRAM (Traffic Light Sequence)

Address	Hex	Meaning
2000	75	MOV direct,#data
2001	90	direct = P1 (90H)
2002	01	RED ON (01H)
2003	02	LJMP
2004	20	High-byte of address 2020H
2005	20	Low-byte of address 2020H
2006	75	MOV direct,#data
2007	90	P1
2008	02	YELLOW ON (02H)
2009	02	LJMP
200A	20	High byte
200B	20	Low byte
200C	75	MOV direct,#data
200D	90	P1
200E	04	GREEN ON (04H)
200F	02	LJMP
2010	20	High byte
2011	20	Low byte
2012	02	LJMP (loop back to start)
2013	20	High byte of 2000H
2014	00	Low byte

DELAY ROUTINE (LOCATED AT 2020H)

Address	Hex	Meaning
2020	7F	MOV R7,#FFH
2021	FF	immediate data
2022	7E	MOV R6,#FFH
2023	FF	immediate data
2024	DE	DJNZ R6,2024
2025	FE	relative offset (-2)
2026	DF	DJNZ R7,2022
2027	FA	relative offset (-6)
2028	22	RET

RESULT:

Hence the traffic light interface was successfully programmed and interfaced with the 8051 microcontrollers, and also the LEDs turned **RED** → **YELLOW** → **GREEN** in a continuous loop with correct delays, demonstrating proper control of the traffic signal using 8051.

Experiment: 7	Sequencing of Hydraulic and Pneumatic circuits.
Date:	

AIM:

To study, design, and demonstrate the **sequential operation** of hydraulic and pneumatic circuits using control valves, actuators (cylinders), and appropriate power sources, ensuring actuators operate in a defined sequence.

Apparatus Required:**Hydraulic System**

SNO	Equipment Name	Quantity
1	Hydraulic power unit (pump + reservoir)	1
2	Double-acting hydraulic cylinders	1
3	Hydraulic hoses	1
4	Direction control valve (4/2 or 4/3 DCV)	1
5	Pressure relief valve	1
6	Flow control valve	1
7	Limit switches / position sensors (optional)	1
8	Mounting board/frame	1

Pneumatic System:

SNO	Equipment Name	Quantity
1	Air compressor	1
2	Double-acting pneumatic cylinders	1
3	Pneumatic hoses / tubes	1
4	3/2 or 5/2 pneumatic DCVs (manual/solenoid/spring return)	1
5	FRL unit (Filter–Regulator–Lubricator)	1
6	Quick exhaust valves (optional)	1
7	Flow control (throttle) valves	1

PROCEDURE:

A. Hydraulic Circuit Sequencing

1. Preparation

1. Connect hydraulic pump to DCV using hoses.
2. Connect double-acting hydraulic cylinder to the valve ports.
3. Ensure pressure relief valve is set within safe operating limits.
4. Remove air from hydraulic lines (bleeding).

2. Circuit Connections

1. Connect DCV to cylinder:
 - Port A → cylinder head side
 - Port B → cylinder rod side
2. Connect flow control valves for speed regulation.
3. Add limit switches (if required) to sense cylinder position.
4. Ensure return line to tank is connected properly.

3. Sequence Setting

Sample sequence: **A+ → B+ → B− → A−**

1. Actuate DCV to extend cylinder A (A+).
2. When A reaches full extension, it activates a limit switch or manually operated valve to trigger cylinder B extension (B+).
3. When B reaches its full stroke, another switch triggers B to retract (B−).
4. After B fully retracts, A retracts (A−).

4. Operation

1. Turn ON hydraulic power unit.
2. Slowly operate the first DCV.
3. Observe the cylinders moving in the defined sequence.
4. Adjust throttling valves to maintain smooth motion.

B. Pneumatic Circuit Sequencing

1. Preparation

1. Connect compressor outlet to FRL unit.
2. Supply clean air to DCV.
3. Connect pneumatic cylinder to DCV using air tubes.

2. Circuit Connections

1. Extend cylinder by supplying air to cap-end port.
2. Retract cylinder by switching DCV to rod-end port.
3. Use one DCV per cylinder or use cascade method for multi-cylinder sequencing.

3. Pneumatic Sequence Example: $A+ \rightarrow B+ \rightarrow B- \rightarrow A-$

1. Manually/solenoid-operated DCV starts extension of cylinder A (A+).
2. When A reaches end, a roller limit valve or 3/2 sensor valve actuates cylinder B (B+).
3. When B fully extends, its end-trigger valve returns B back (B-).
4. When B retracts fully, the final valve sends signal to retract A (A-).

4. Operation

1. Turn ON compressor.
2. Supply air to the DCV.
3. Operate first valve to start the sequence.
4. Observe smooth sequential movement of actuators.
5. Adjust speed using pneumatic flow control valves.

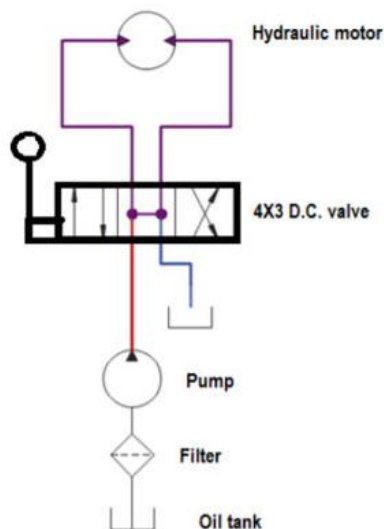
Connections 1.

1. When the 4/3 valve is in its neutral position (tandem design), the hydraulic motor is hydraulically locked and the pump is unloaded back to the tank.
2. When the 4/3 valve is actuated into the flow path, the hydraulic motor is rotates clockwise direction against its load as oil flows from port P through port A. Oil in the hydraulic motor is free to flow back to the tank through the four-way valve from port B through port T.

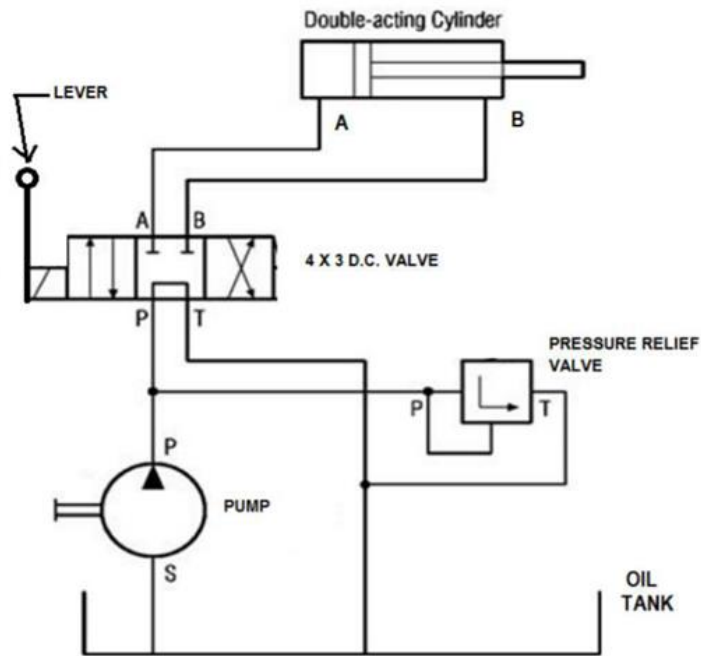
3. When the 4/3 valve is actuated into the right-envelope configuration, the motor rotates anticlockwise direction as oil flows from port P through port B. Oil in the blank end is returned to the tank via the flow path from port A to port T. Hence we get rotary motion of hydraulic motor in form of mechanical energy

Procedure

1. Switch on the three phase connection given to Induction motor
2. Rotate pressure relief valve anticlockwise direction for two minutes
3. By observing the pressure gauge of pressure line adjust pressure between 12 to 15 kgf/cm²
4. Check oil level in tank to be full shown by indicator
5. Observe the reciprocating motion of double acting cylinder and hydraulic motor by varying the Pressure, using pressure knob.
6. Observe the rotary motion of hydraulic motor.



Working of double acting cylinder and hydraulic motor



Basic hydraulic circuit for double acting cylinder

Result:

Hydraulic and pneumatic sequencing circuits were successfully constructed and tested. The actuators operated in the desired order according to the pre-defined sequence ($A^+ \rightarrow B^+ \rightarrow B^- \rightarrow A^-$). The experiment demonstrated the working principles of directional control valves, actuators, pressure/flow control, and signal-based sequencing in both hydraulic and pneumatic systems. Proper sequencing ensured safe, smooth, and coordinated actuator motion for industrial automation applications.

Expt. No.7(i) ACTUATION OF SINGLE ACTING CYLINDER USING3/2 SINGLE SOLENOID
VALVE (ELECTRO -
PNEUMATIC)**AIM:**

To construct an electro - pneumatic circuit to actuate the single acting cylinder using 3/2 single solenoid valve

Apparatus required:

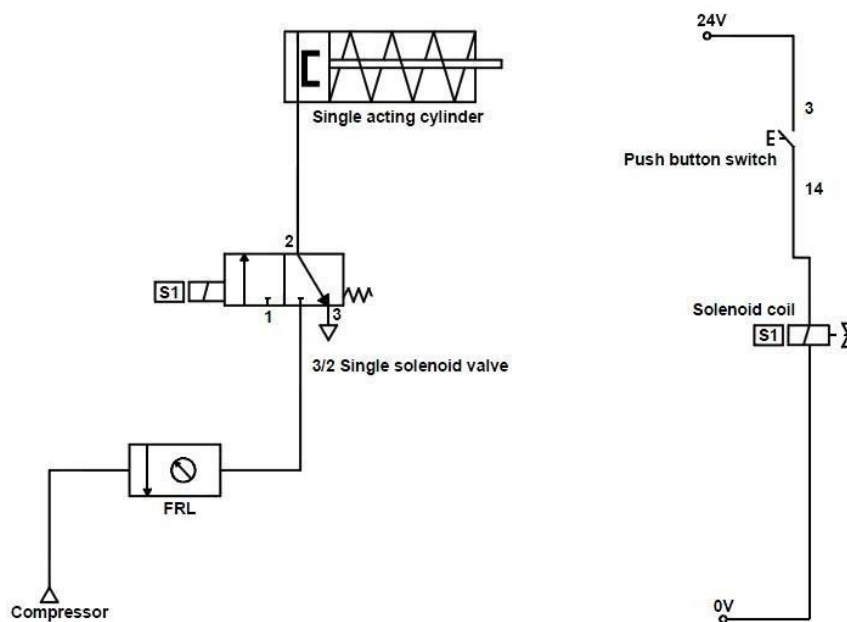
1. Compressor
2. FRL unit
3. 3/2 Single solenoid valve
4. Electrical trainer
5. Single acting cylinder
6. Batch card

Procedure:

1. Draw the circuit diagram.
2. The electrical trainer gives voltage to pneumatic panel.
3. Connect the air supply to FRL unit.

4. The output of FRL unit is connected to the 3/2 single solenoid valve which is activated by the electrical trainer.
5. The output of the 3/2 single solenoid valve is connected to the single acting cylinder.
6. Observe the actuation of the single acting cylinder using 3/2 single solenoid valve.

Circuit diagram:



Result:

Thus the actuation of single acting cylinder was carried out using 3/2 single solenoid valve

Expt. No.7(ii) ACTUATION OF SINGLE ACTING CYLINDER
USING 'ON' DELAY TIMER

AIM:

To construct an electro pneumatic circuit for the actuation of single acting cylinder using ON delay timer

Apparatus required:

1. Compressor
2. FRL unit
3. Time delay valve
4. Electrical controller
5. Single acting cylinder
6. 3/2 Single solenoid valve
7. Batch card

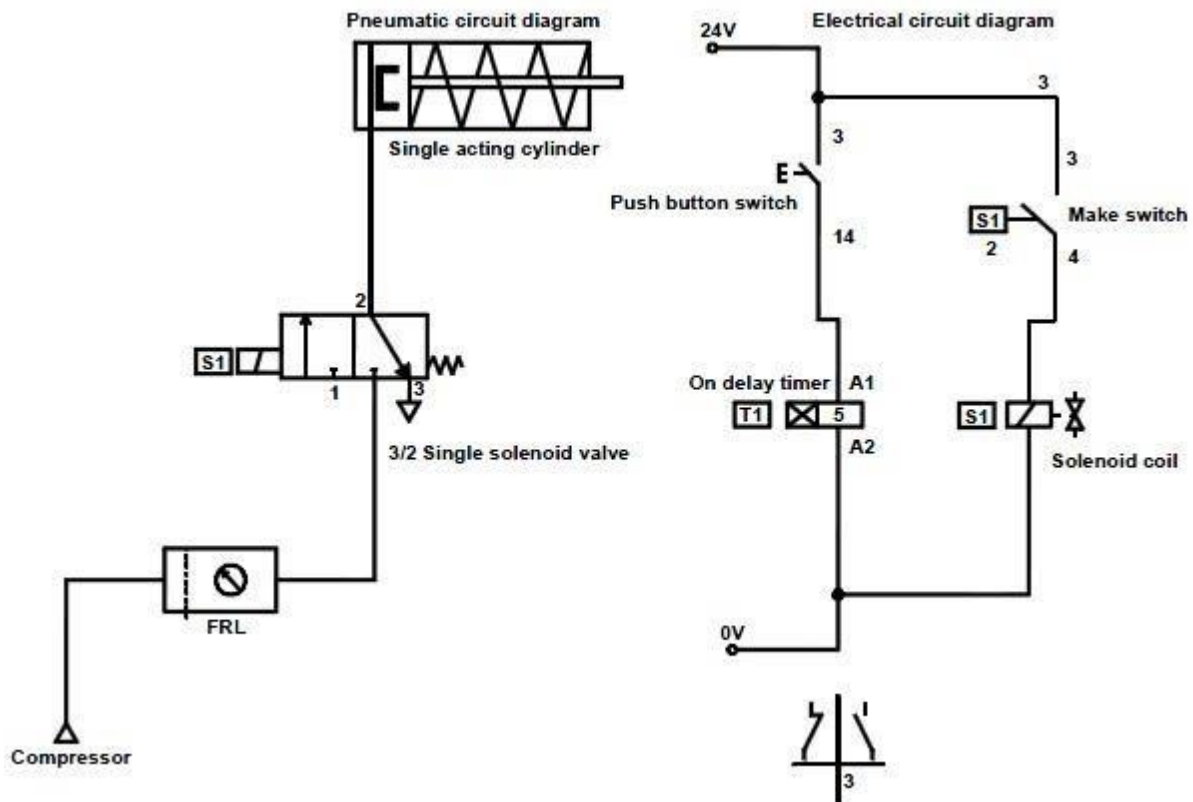
Procedure:

1. The power supply is provided to electrical controller by interfacing the +ve to +ve and -ve to -ve.
2. The power supply is provided to pneumatic trainer from electrical controller by interfacing 24 +ve to +ve and -ve to -ve.
3. Using the SPDT switch energize the corresponding solenoid to get the desired movement of the cylinder.
4. Actuate the time delay circuit.
5. From the time delay, give connection to single acting cylinder according to the time set.
6. Design and draw the pneumatic circuit.

7. Test the circuit.

8. Observe the working of the cylinder.

Circuit diagram:



Result:

Thus the actuation of single acting cylinder was carried out using ON delay timer.

Expt. No.7(iii) ACTUATION OF SINGLE ACTING CYLINDER
USING 'OFF' DELAY TIMER

AIM:

To construct an electro pneumatic circuit for the actuation of single acting cylinder using OFF delay timer.

Apparatus required:

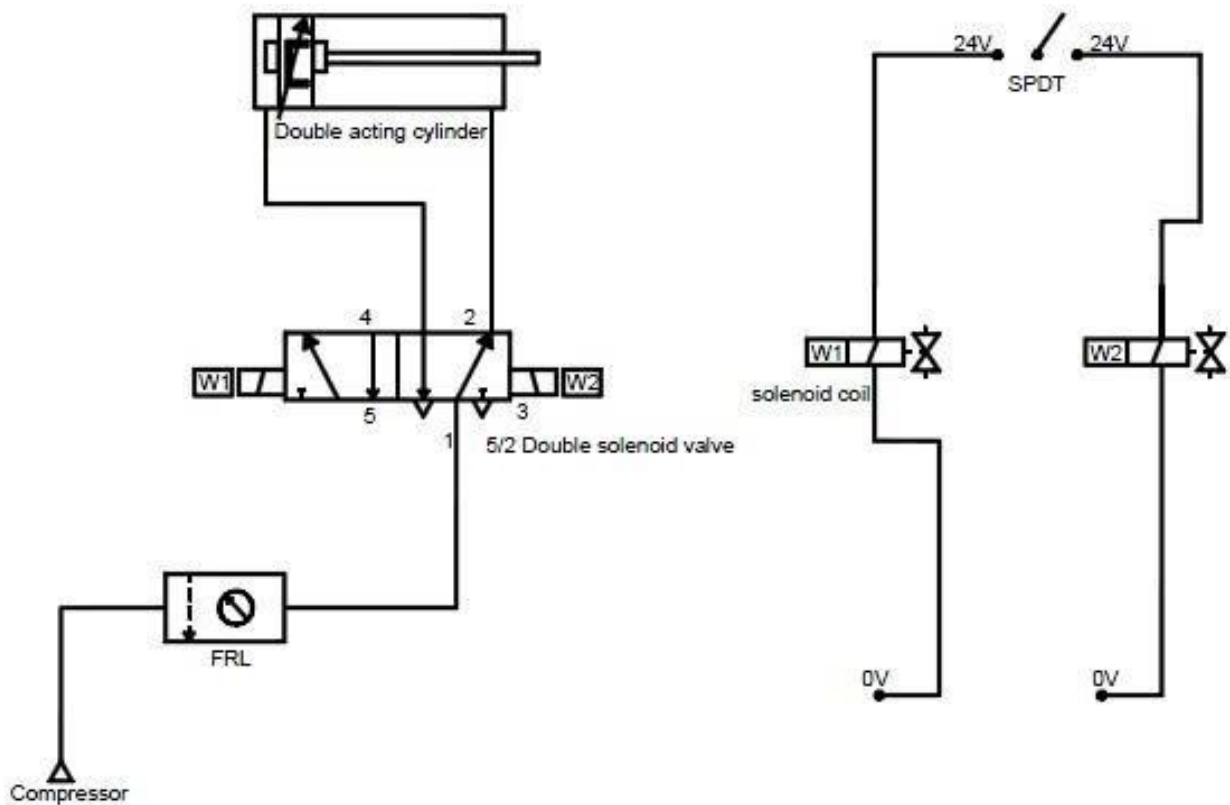
1. Compressor
2. FRL unit
3. 3/2 Single acting cylinder
4. Electrical controller
5. Single acting cylinder
6. OFF delay timer
7. Batch card

Procedure:

1. Provide power supply to pneumatic trainer from electrical controller by inter facing 24+ and 24-.
2. Provide 24V power supply to timer.
3. Any one of the output of FRL unit direct connect to 3/2 single solenoid valve.
4. Single solenoid valve output is connecting to single acting cylinder.
5. Give +24V and -24V in Timer.
6. Output of Timer connected to solenoid coil.

7. Check the all circuit.
8. Observe the working of cylinder.
9. Observe the working circuit.

Circuit diagram:



Result:

Thus the movement of single acting cylinder was carried out using OFF delay timer.

MODELING AND ANALYSIS OF BASIC HYDRAULIC, PNEUMATIC AND ELECTRICAL CIRCUITS USING PNEUMOSIM SOFTWARE

Expt. No.8 (a) SIMULATION OF DOUBLE ACTING CYLINDER USING TIME DELAY VALVE

AIM:

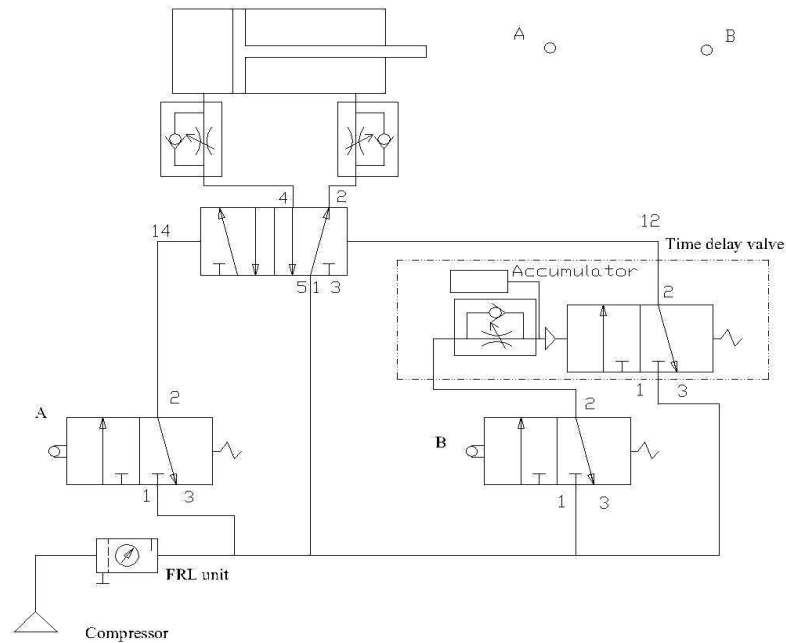
To design a pneumatic circuit to actuate a double acting cylinder using time delay valve and simulate the circuit in Pneumosim software

Apparatus required:

1. Flow control valve
2. Time delay valve
3. 3/2 way spring operated valve
4. 5/2 way valve

Pneumosim software Procedure:

- The circuit is designed as per the given experiment.
- Open a new file from the pull down menu bar.
- Select and drag the required components from the components library.
- Connect the components as per the circuit connection.
- Simulate the circuit to check the actuation.
- Observe the actuation of double acting cylinder using time delay valve.

Circuit diagram:**Result:**

Thus the circuit is designed and simulated by using Pneumosim software for the given application

Expt. No.8 (b) SIMULATION OF DOUBLE ACTING CYLINDERS IN THE SEQUENCE OF A+ B+ B- A-

AIM:

To simulate double acting cylinders in the sequence of A+ B+ B- A- using

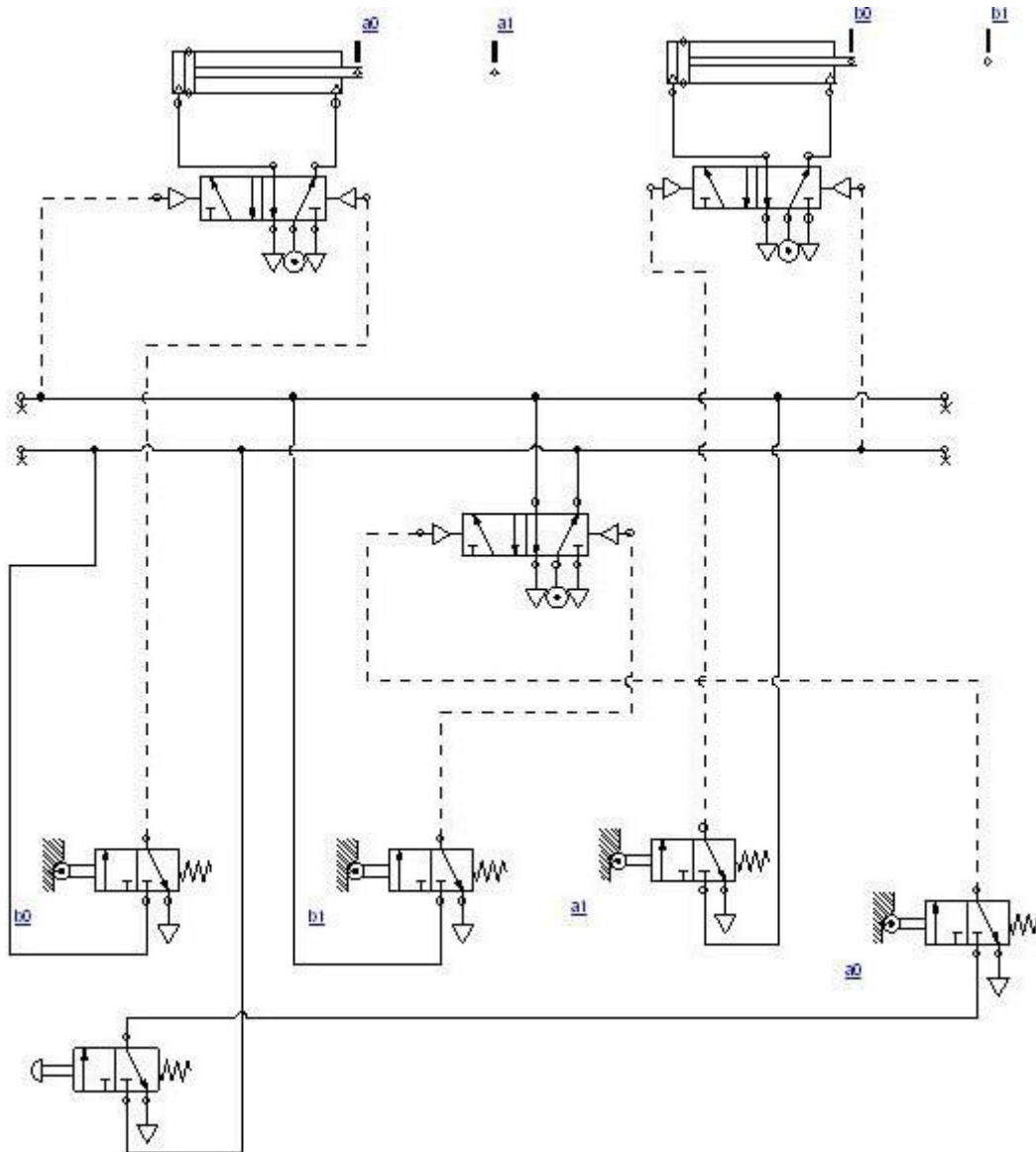
Pneumosim software Apparatus required:

1. Double acting cylinders
2. 5/2 double solenoid valve
3. Compressor
4. Limit Switches
5. Push button valves

Pneumosim Software Procedure:

1. The circuit is designed as per the given experiment.
2. Open a new file from the pull down menu bar.
3. Select and drag the required components from the components library.
4. Connect the components as per the circuit connection.
5. Simulate the circuit to check the actuation.

Circuit diagram:



Result:

Thus the simulation of double acting cylinders in the sequence of A+ B+ B- A- was achieved using Pneumosim software.

STUDY OF PLC AND ITS APPLICATIONS

Expt. No.8 (c) ACTUATION OF SINGLE ACTING CYLINDER WITH
PLC USING 'AND' GATE

AIM:

To actuate the single acting cylinder using AND gate with PLC ladder diagram

Apparatus required:

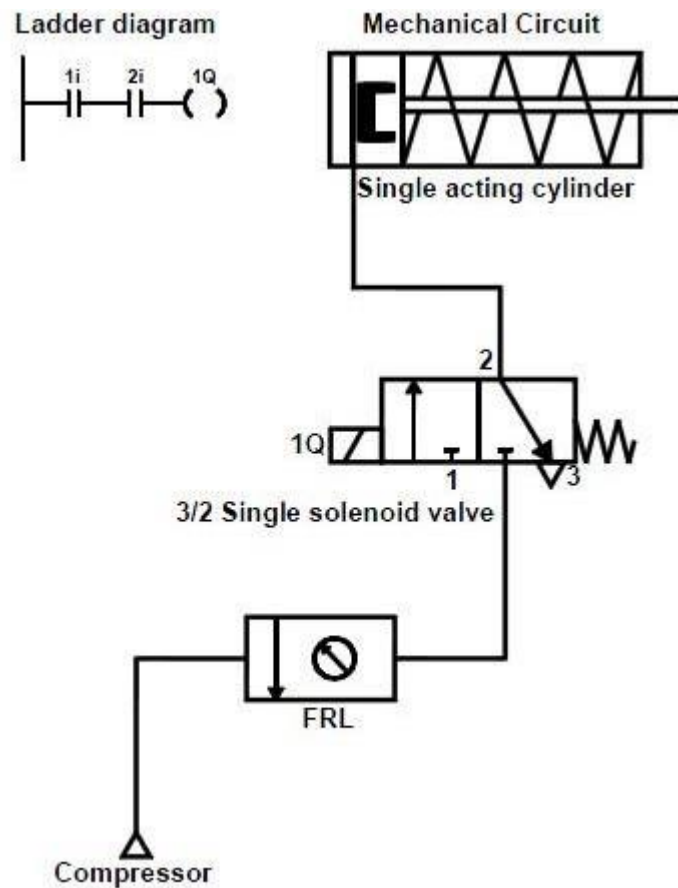
1. Compressor
2. FRL unit
3. Single acting cylinder
4. PLC
5. Versa Pro software
6. 3/2 single solenoid

valve Procedure:

1. Draw the circuit diagram.
2. Provide +24 V and -24 V from PLC trainer to electro pneumatic panel.
3. The output of the PLC (Q1) is connected to the input of 3/2 single solenoid valve.
4. Then open the Versa Pro software in desktop.
5. Interface the PLC with the system using RS232 cable.
6. Connect the air supply to FRL unit.

7. The output of FRL unit is connected to the 3/2 single solenoid valve.
8. Check the Ladder diagram.
9. Run the PLC. When two inputs (1i, 2i) are high, then output also will be high.

Circuit diagram:



Result:

Thus the actuation of single acting cylinder with PLC using AND Gate was done.

Expt. No.8 (d) ACTUATION OF DOUBLE ACTING CYLINDER WITH PLC USING 5/2 SINGLE SOLENOID VALVE

AIM:

To actuate the double acting cylinder with PLC using 5/2 single solenoid valve.

Apparatus required:

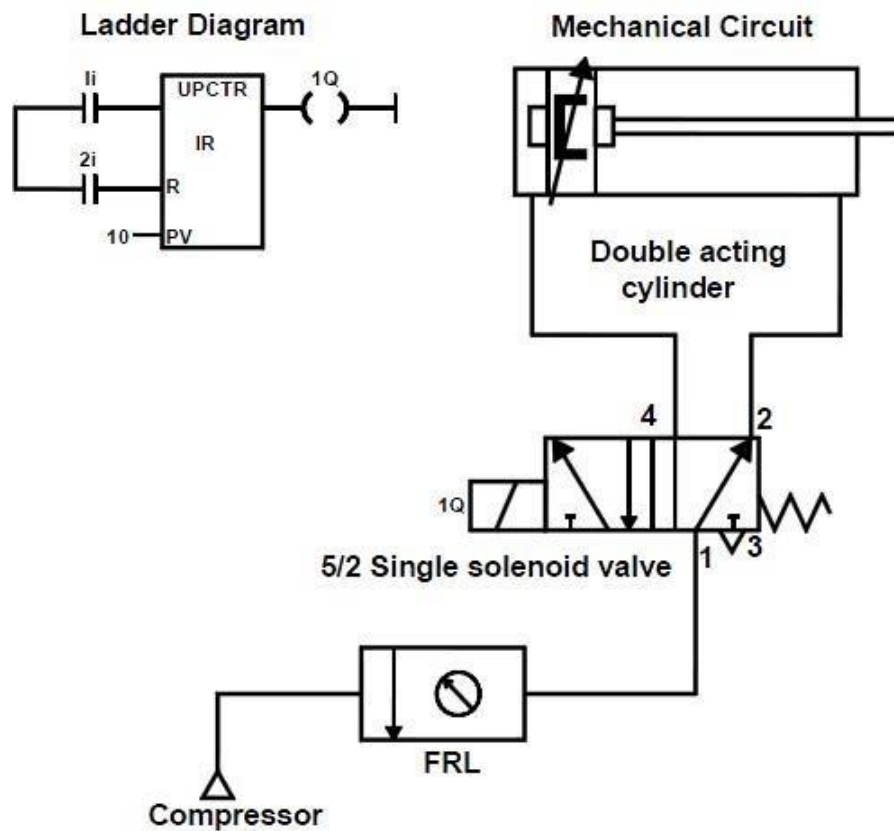
1. Compressor
2. FRL unit
3. 5/2 single solenoid valve
4. PLC
5. Double acting cylinder
6. Versa Pro software

Procedure:

1. Draw the circuit diagram.
2. Provide +24 V and -24 V from PLC trainer to pneumatic panel kit.
3. Open the Versa Pro software in desktop.
4. Interface the PLC with PC using RS232 cable.
5. Draw the ladder diagram.

6. The output of PLC (q1) is connected to the input of solenoid valve.
7. Check the ladder diagram.
8. Connect the air supply to FRL unit.
9. Run the PLC input (1i), switch continuously ON and OFF. When it reaches the preset value (pv), the cylinder gets actuated.

Circuit diagram:



Result:

Thus the double acting cylinder is actuated with 5/2 single solenoid valve using PLC.

Expt. No.8 (e) AUTOMATIC ACTUATION OF SINGLE ACTING CYLINDER USING PLC

AIM:

To simulate the automatic sequence of single acting cylinder by using PLC

Apparatus required:

1. Compressor
2. FRL unit
3. 3/2 single solenoid valve
4. PLC
5. Single acting cylinder
6. Versa Pro

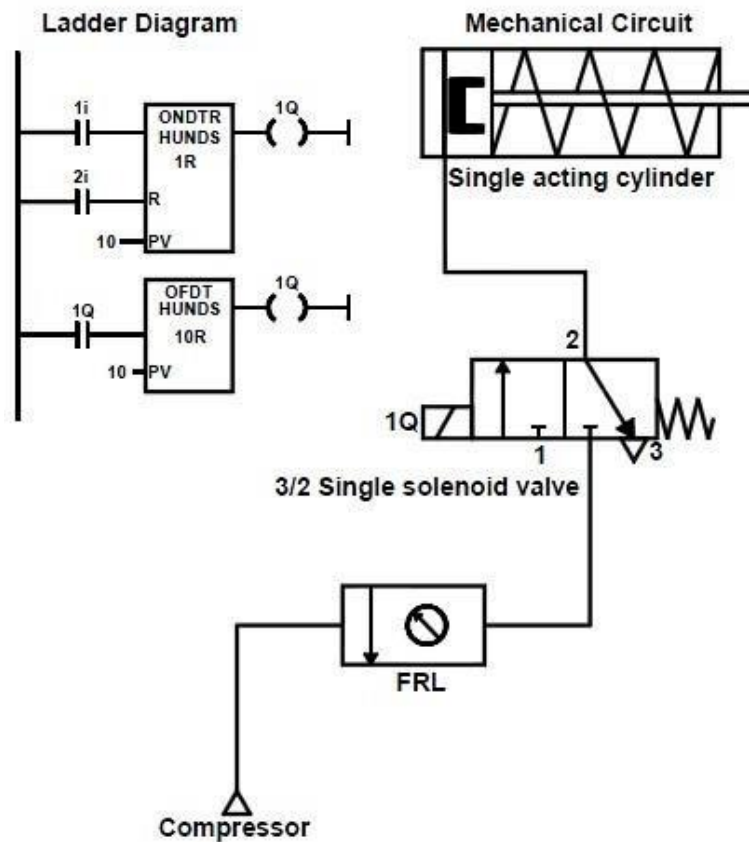
Software

Procedure:

1. Draw the circuit diagram.
2. Provide +24 V and –24 V from PLC trainer to pneumatic panel kit.
3. Open the Versa Pro software in desktop.
4. Interface the PLC with PC using RS232 cable.
5. Draw the ladder diagram.

6. The output of PLC (q1) is connected to the input of solenoid valve.
7. Check the ladder diagram.
8. Connect the air supply to FRL unit.
9. Run the PLC. Observe the automatic reciprocation of single acting cylinder.

Circuit diagram:



Result:

Thus the ladder diagram for the automatic running of single acting cylinder is drawn and executed.

Experiment: 9	Vision based image acquisition and processing technique for inspection and classification.
Date:	

AIM:

To design and implement a vision-based image acquisition and processing system using MATLAB to inspect and classify objects based on specific features, enhancing quality control in industrial applications.

Apparatus Required:

1. Personal Computer
2. MATLAB

Procedure:

Open MATLAB

Open new M-file

Type the program

Save in current directory

Compile and Run the program

For the output see command window / Figure window

Program:

% Step 1: Image Acquisition

% Using a pre-captured image for demonstration.

Replace with cam acquisition if needed

originalImage = imread('image.jpg'); % Replace 'image.jpg' with your image path

% Step 2: Image Processing

% Convert to grayscale

grayImage = rgb2gray(originalImage);

% Apply Gaussian filter

filteredImage = imgaussfilt(grayImage, 2);

% Edge detection using Canny

edges = edge(filteredImage, 'Canny');

% Step 3: Feature Extraction

```
% Perform morphological operations to clean edges
cleanedEdges = imdilate(edges, strel('disk', 2));
cleanedEdges = imerode(cleanedEdges, strel('disk', 2));
% Find connected components
[labels, numObjects] = bwlabel(cleanedEdges);
measurements = regionprops(labels, 'Area', 'BoundingBox', 'Centroid');
```

% Step 4: Classification (Simple Area-based Classification)

```
classificationResults = cell(numObjects, 1);
for i = 1:numObjects
    area = measurements(i).Area;
    if area < 500
        classificationResults{i} = 'Small Object';
    elseif area < 2000
```

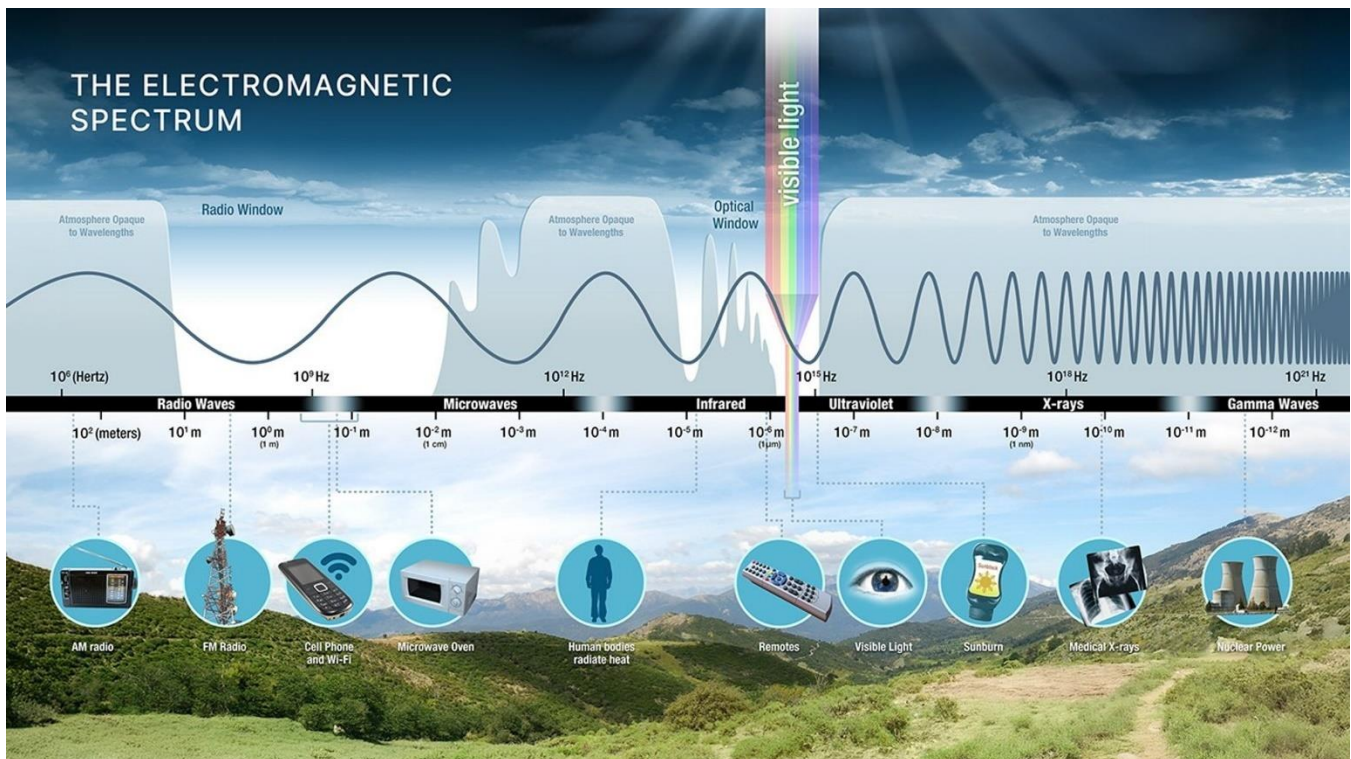
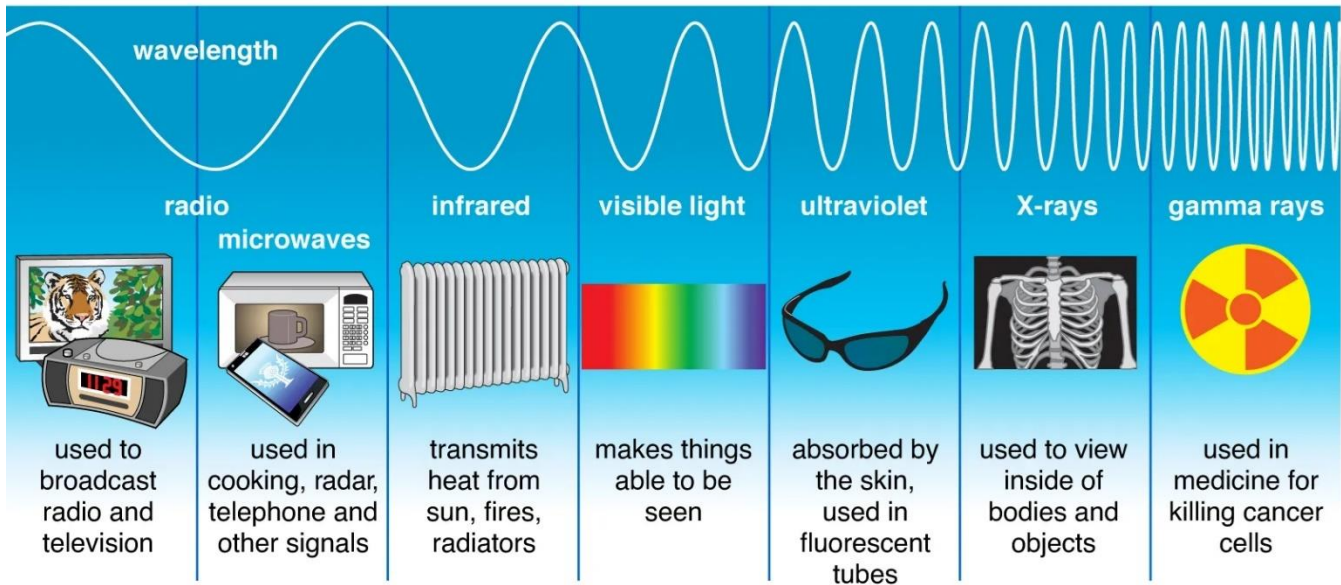
```
        classificationResults{i} = 'Medium Object';
    else
        classificationResults{i} = 'Large Object';
    end
end
```

% Step 5: Display Output

```
figure;
imshow(originalImage);
hold on;
% Draw bounding boxes and display classifications
for i = 1:numObjects
    bbox = measurements(i).BoundingBox;
    rectangle('Position', bbox, 'EdgeColor', 'r', 'LineWidth', 2);
    text(measurements(i).Centroid(1), measurements(i).Centroid(2), ...
        classificationResults{i}, 'Color', 'blue', 'FontSize', 12, 'FontWeight', 'bold');
end
```

```
title('Inspection and Classification Result');
hold off;
```

Types of Electromagnetic Radiation





Result:

The system successfully captures images from the camera and processes them to identify and classify objects based on predefined features

Experiment: 10	Familiarization with Concept of IoT and its Open-Source Microcontroller / SBC
Date:	

AIM:

To study and understand the concept of Internet of Things (IoT) and to become familiar with commonly used open-source microcontrollers and Single Board Computers (SBCs) such as Arduino, ESP8266/ESP32, and Raspberry Pi.

Apparatus:

SNO	Equipment Name	Quantity
1	Arduino UNO / Arduino Nano (open-source microcontroller)	1
2	ESP8266 Node MCU / ESP32 Wi-Fi microcontroller	1
3	Raspberry Pi SBC (any model)	1
4	USB cable for programming	1
5	Power supply / adapter	1
6	Breadboard and jumper wires	1
7	Sensors (Temperature sensor / Ultrasonic sensor – optional)	1
8	Laptop/PC with Arduino IDE or Python installed	1
9	Wi-Fi network (for IoT demonstration)	1

THEORY:

IoT (Internet of Things) refers to interconnecting physical devices over the internet to collect and exchange data.

IoT systems consist of:

1. Sensors – measure physical data
2. Microcontrollers / SBCs – process and send data
3. Communication modules – Wi-Fi, Bluetooth, LoRa, GSM
4. Cloud Platforms –Blynk Clout, Thing-speak, Firebase, MQTT servers
5. Actuators – control output devices

Open-source controllers widely used in IoT:

- Arduino → easy to program, ideal for sensors
- ESP8266/ESP32 → built-in Wi-Fi, suited for IoT
- Raspberry Pi → Linux-based SBC for advanced IoT systems

Procedure:**1. Understanding IoT Components**

- a) Identify sensors (temperature, IR, ultrasonic).
- b) Study actuators (LED, relay, motor).
- c) Observe Wi-Fi modules (ESP8266/ESP32).

2. Familiarization with Open-Source Microcontrollers / SBCs**1. Observe the Arduino board and identify:**

- Digital pins
- Analog pins
- Power pins
- Reset switch
- USB port

2. Observe Node MCU/ESP32 board and identify:

- Wi-Fi/BT module
- GPIOs
- Micro-USB port

3. Observe Raspberry Pi SBC and identify:

- GPIO pins
- USB ports
- HDMI port
- Ethernet/Wi-Fi

3. Connecting and Testing IoT Boards

1. Connect Arduino/Node MCU to a PC using USB cable.
2. Open Arduino IDE.
3. Select correct board and COM port.
4. Upload a sample IoT program (LED blink or Wi-Fi connect).

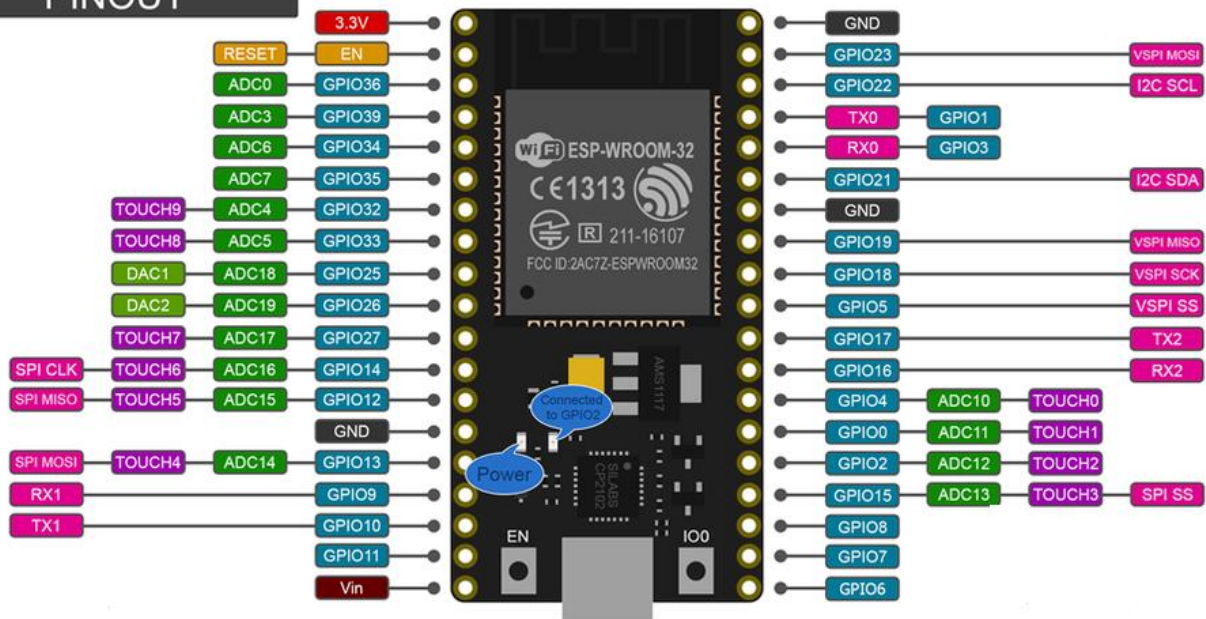
4. Basic IoT Demonstration

1. Connect Node MCU/ESP32 to Wi-Fi.
2. Read a sensor value.
3. Send data to cloud (e.g., Blynk cloud, Thingspeak) or print on serial monitor.
4. Observe data being updated online.

Pin Diagram:

NodeMCU-32S

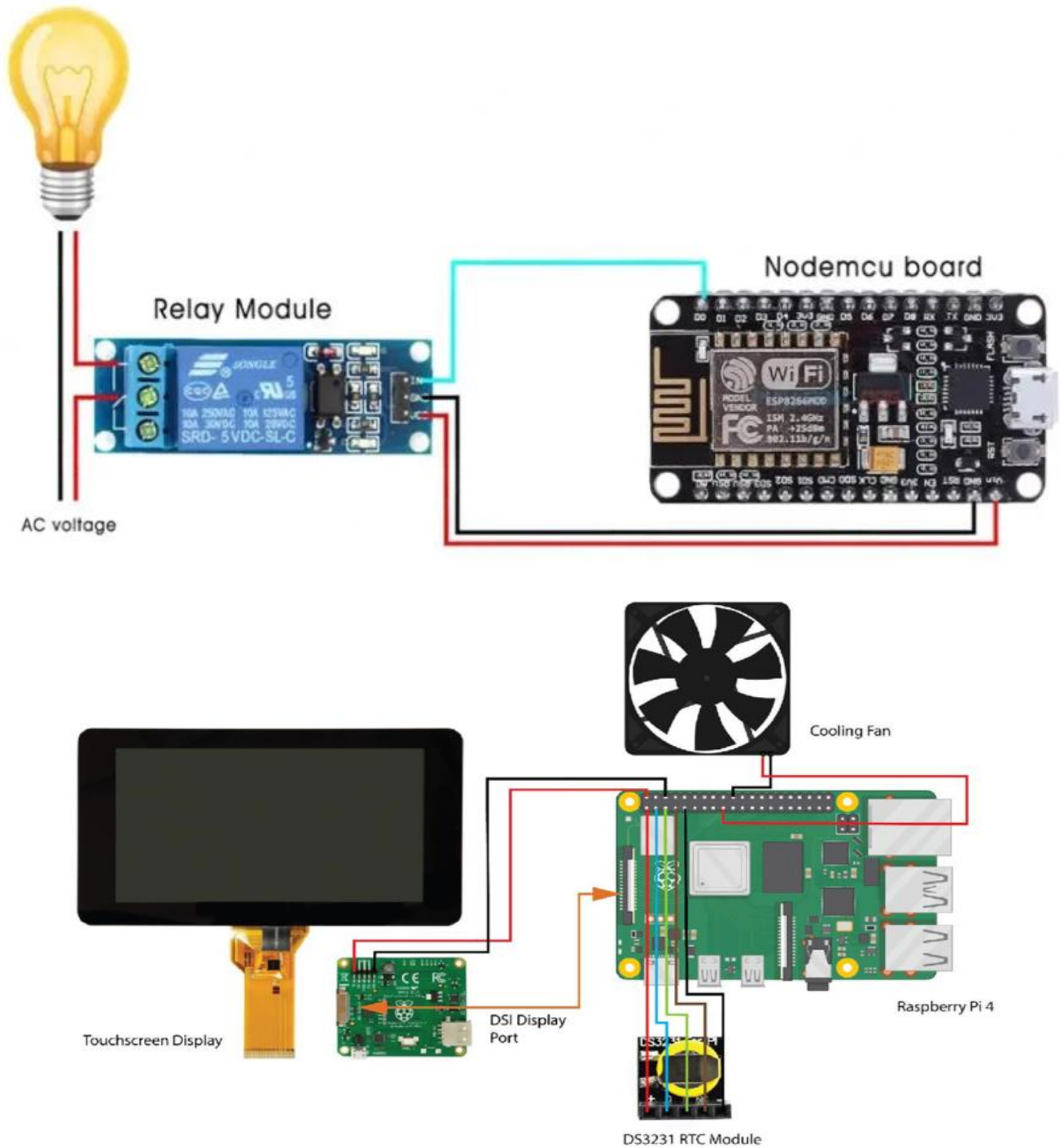
PINOUT



IoT TEST PROGRAM (ESP8266 LED Blink)

```
void setup() {
  pinMode(2, OUTPUT); // Built-in LED
}

void loop() {
  digitalWrite(2, LOW); // LED ON
  delay(1000);
  digitalWrite(2, HIGH); // LED OFF
  delay(1000); }
```

Circuit Diagram:**Figure 2. Schematics diagram of wiring connection to the Raspberry Pi.**

Raspberry Pi Pin Diagram:

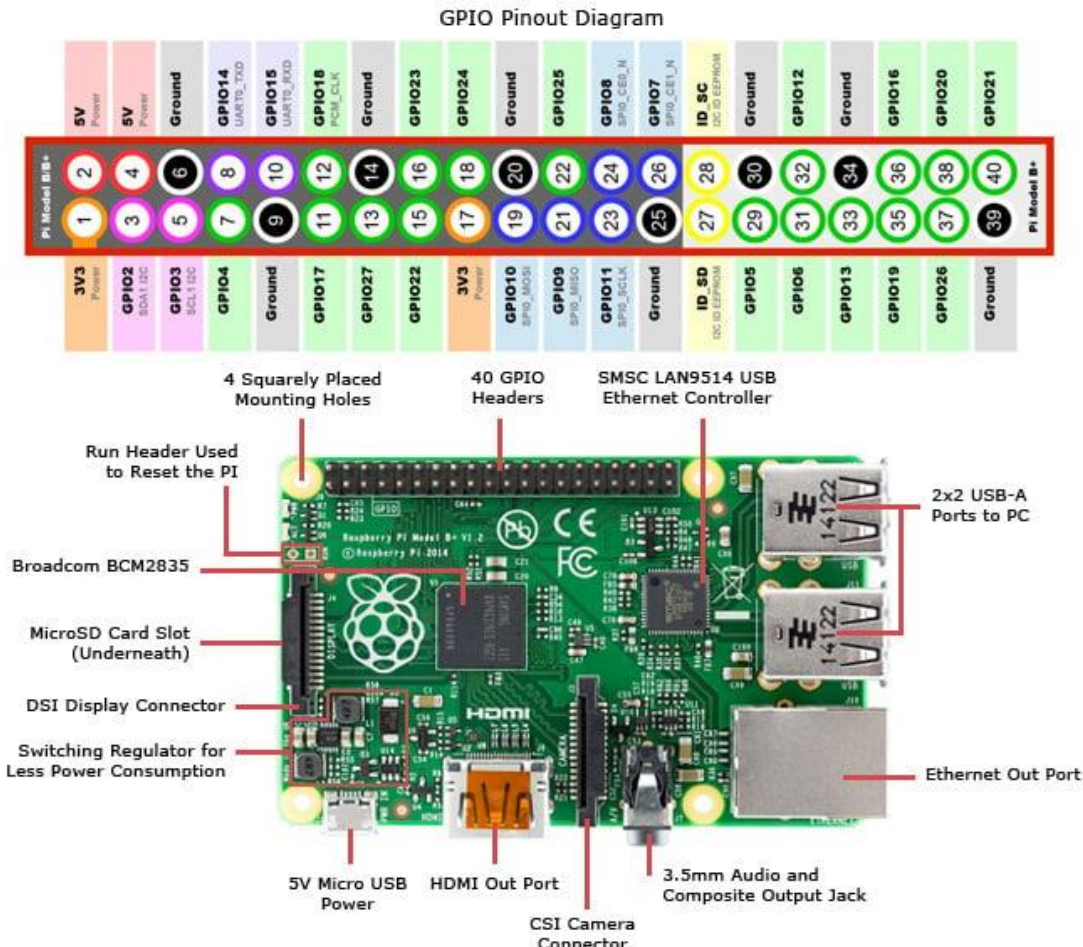


Figure 3. Schematics diagram of wiring connection to the Raspberry Pi

RESULT:

Hence the concept of IoT was successfully studied. The working principles of open-source IoT hardware such as **Arduino**, **ESP8266/ESP32**, and **Raspberry Pi** were understood, including their pin functions, connectivity, and basic programming, and also A basic IoT demonstration (LED blinking / Wi-Fi connection) was performed, showing how IoT devices operate and communicate.

Experiment: 11	Write a program to turn ON/OFF motor using microcontroller/SBC through internet.
Date:	

AIM:

To control a DC motor using a NodeMCU (ESP8266) via the internet through Blynk Cloud, demonstrating IoT-based remote motor control.

Apparatus Required:

SNO	Equipment Name	Quantity
1	NodeMCU ESP8266 Board	1
2	DC Motor	1
3	Motor Driver Module (L293D or L298N)	1
4	USB Cable for NodeMCU	1
5	Breadboard & Jumper Wires	1
6	Arduino IDE Installed on PC	1
7	Blynk App installed on a smartphone	1
8	Wi-Fi Network (Internet)	1
9	Power Supply (5V for motor & 3.3V for NodeMCU)	1

Procedure:**Hardware Setup**

1. Connect **DC Motor** to **L293D motor driver outputs (OUT1 & OUT2)**.
2. Connect **L293D input pins** to NodeMCU GPIOs (e.g., D1 → IN1, D2 → IN2).
3. Connect **EN pin of L293D** to 5V for motor enable.
4. Connect **NodeMCU GND** to motor driver GND.
5. Connect **motor power supply** (e.g., 5V–12V) to L293D VCC.
6. Connect **NodeMCU to PC** via USB.

2. Software Setup

1. Install Arduino IDE.
2. Install ESP8266 board via Board Manager.
3. Install Blynk library in Arduino IDE.
4. Open Blynk App on your smartphone.
5. Create a new project:
 - Device: NodeMCU
 - Connection: Wi-Fi
 - Add a Button widget to control motor
 - Copy the Auth Token (sent to email)

3. Programming (Arduino IDE)

```
#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

// Replace with your Wi-Fi credentials
char ssid[] = "YOUR_WIFI_SSID"; //Enter your Username of Hotspot
char pass[] = "YOUR_WIFI_PASSWORD"; //Enter your Hotspot password

// Blynk Auth Token from app or Blynk Cloud
char auth[] = "YOUR_BLYNK_AUTH_TOKEN";

// Motor driver pins
#define IN1 D1
#define IN2 D2

void setup()
{
  Serial.begin(115200);
  Blynk.begin(auth, ssid, pass);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
}
```

```
// Ensure motor is off initially
digitalWrite(IN1, LOW);
digitalWrite(IN2, LOW);
}

// Blynk button V1 mapped
BLYNK_WRITE(V1)
{
  int pinValue = param.asInt(); // Get button state

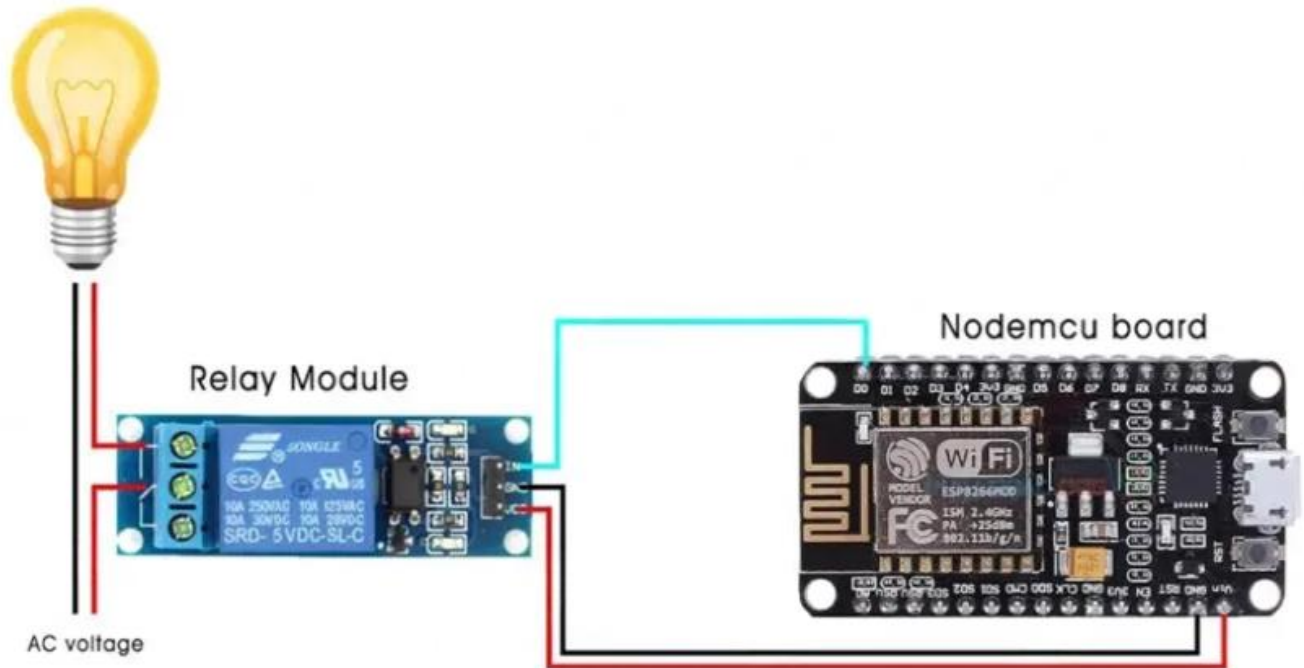
  if(pinValue == 1) // Button pressed → motor ON
  {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
  }
  else // Button released → motor OFF
  {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
  }
}

void loop()
{
  Blynk.run();
}
```

4. Upload & Test

- ✓ Connect NodeMCU to PC via USB.
- ✓ Select Board → NodeMCU 1.0 and correct COM Port.
- ✓ Upload program from Arduino IDE.
- ✓ Open Blynk App on smartphone.
- ✓ Press the Button widget to turn motor ON/OFF.

Output Circuit Diagram:



RESULT:

Hence the DC motor can now be remotely controlled over the internet using **Node MCU and Blynk Cloud**. Pressing the **Blynk button ON** activates the motor, and OFF stops it. This demonstrates **IoT-based motor control** using an open-source microcontroller and smartphone application.

Experiment: 12	Write a program to interface sensors to display the data on the screen through internet.
Date:	

AIM:

To interface a sensor with an ESP32 microcontroller and display the sensed data (temperature and humidity) on a web page accessible through the Internet.

Apparatus Required:

SNO	Equipment Name	Quantity
1	ESP32 Development Board	1
2	DHT11/DHT22 Temperature & Humidity Sensor	1
3	Jumper Wires	1
4	USB Cable (for programming ESP32)	1
5	Wi-Fi Internet Connection	1
6	Arduino IDE with ESP32 Board Support Installed	1

PROCEDURE:**1. Hardware Connections**

- Connect DHT11 VCC → 3.3V of ESP32
- Connect DHT11 GND → GND of ESP32
- Connect DHT11 Data pin → GPIO 4 of ESP32
- Place a 10k pull-up resistor between DATA and VCC (optional based on module)

2. Software Setup

1. Install Arduino IDE.

Install ESP32 board:

File → *Preferences* → *Additional Board Manager URLs* → add:

https://dl.espressif.com/dl/package_esp32_index.json

2. Install DHT sensor library:

Sketch → *Include Library* → *Manage Libraries* → search “DHT sensor library”

3. Write Program

- Program ESP32 to read sensor data.
- Start a simple web server.
- Display sensor data on a webpage.

4. Upload Program

- Select board: ESP32 Dev Module
- Select correct COM port
- Upload the code to ESP32

5. Display Data

After reset, open Serial Monitor.

Note the IP address of ESP32.

Enter this IP in any browser → The sensor data will display on screen.

CODE (ESP32 + DHT11 Web Server)

```
#include <WiFi.h>
```

```
#include "DHT.h"
```

```
#define DHTPIN 4    // Data pin connected to GPIO 4
```

```
#define DHTTYPE DHT11 // DHT11 sensor
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
// Enter Wi-Fi credentials
```

```
const char* ssid = "YOUR_WIFI_NAME"; // Use your Hotspot user id
```

```
const char* password = "YOUR_WIFI_PASSWORD"; // Use your hotspot password
```

```
WiFiServer server(80);
```

```
void setup() {
  Serial.begin(115200);
  dht.begin();

  // Connect to Wi-Fi
  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }

  Serial.println("\nWiFi connected!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  server.begin();
}

void loop() {
  WiFiClient client = server.available();
  if (!client) return;

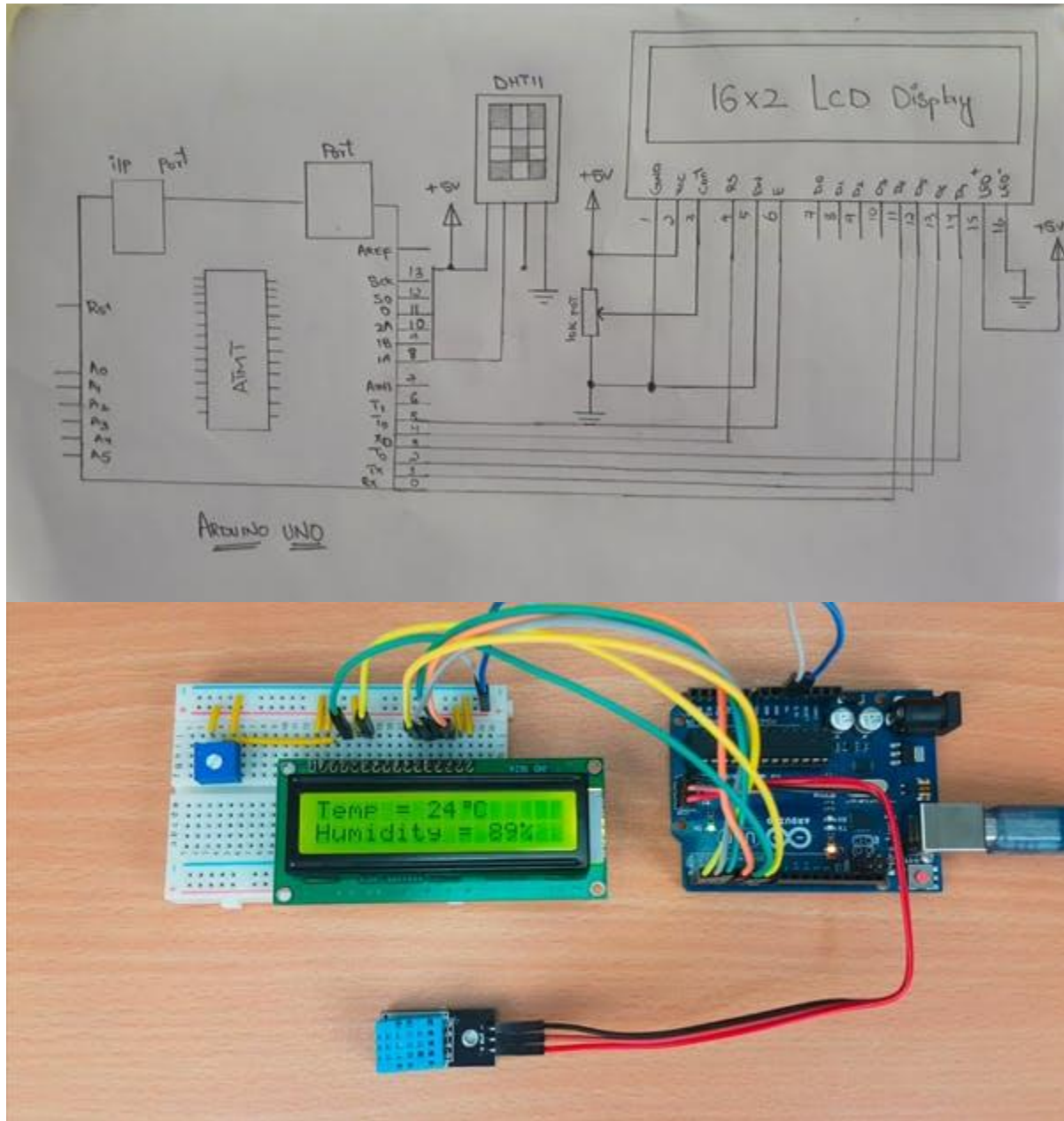
  String request = client.readStringUntil('\r');
  client.flush();

  float h = dht.readHumidity();
  float t = dht.readTemperature();

  // Prepare webpage
  String html = "<html><body style='font-family:Arial; text-align:center;'>";
  html += "<h1>IoT Sensor Data</h1>";
  html += "<h2>Temperature: " + String(t) + " °C</h2>";
  html += "<h2>Humidity: " + String(h) + " %</h2>";
  html += "<p>Data updates every time you refresh the page.</p>";
  html += "</body></html>";

  // Send response
  client.println("HTTP/1.1 200 OK");
  client.println("Content-type:text/html");
  client.println();
  client.println(html);
  client.stop();
}
```


Setup the Hardware Circuit Diagram or Pic of Hardware connection:



RESULT:

The ESP32 successfully reads temperature and humidity values from the DHT11 sensor and displays the data through a web page over the Internet. By connecting to the ESP32's IP address using a browser, the user can view real-time sensor readings remotely.

Experiment: 13	Interface the sensors with microcontroller/SBC and write a program to turn ON/OFF Solenoid valve through internet when sensor data is detected.
Date:	

AIM:

To interface a sensor with a microcontroller (ESP32) and write a program that automatically turns **ON/OFF a solenoid valve through the Internet** based on sensor readings and remote commands.

Apparatus Required:

SNO	Equipment Name	Quantity
1	ESP32 Development Board	1
2	Soil Moisture Sensor (or any sensor as per requirement)	1
3	12V Solenoid Valve	1
4	Relay Module (5V) for solenoid valve control	1
5	12V Power supply for solenoid	1
6	Jumper wires	1
7	USB cable for programming	1
8	Arduino IDE with ESP32 board support	1
9	Wi-Fi Internet connection	1
10	ESP32 Development Board	1

PROCEDURE:**1. Hardware Connections****Soil Moisture Sensor → ESP32**

- VCC → 3.3V
- GND → GND
- Analog Out (A0) → GPIO 34 (Analog pin)

Solenoid Valve (through Relay)

- Relay IN → GPIO 5
- Relay VCC → 5V, GND → GND
- Solenoid valve:
 - One wire → 12V supply
 - Other wire → Relay NO (Normally Open terminal)
 - Relay COM → 12V supply ground return

2. Software Setup

1. Install **Arduino IDE**.
2. Install ESP32 board support.
3. Connect ESP32 via USB.
4. Write the program to:
 - Read sensor data
 - Create a Web Server
 - Control solenoid valve ON/OFF via webpage buttons
 - Auto-activate valve based on sensor reading

3. Program Upload

- Select Board: ESP32 Dev Module
- Select correct COM Port
- Upload the program

4. Testing

1. Open Serial Monitor → note ESP32 IP address.
2. Open browser → enter IP.
3. View:
 - Sensor readings
 - ON/OFF buttons for valve
4. Solenoid will also auto-turn ON/OFF depending on sensor values.

CODE (ESP32 + Sensor + Solenoid Control via Web Page)

```
#include <WiFi.h>
```

```
const char* ssid = "YOUR_WIFI_NAME"; //Enter your Hotspot User id  
const char* password = "YOUR_WIFI_PASSWORD"; // Enter your Hotspot password
```

```
#define SENSOR_PIN 34    // Soil moisture sensor analog pin  
#define RELAY_PIN 5      // Relay control pin for solenoid valve  
int threshold = 2000;    // Moisture threshold (adjust as needed)
```

```
WiFiServer server(80);
```

```
void setup() {  
  Serial.begin(115200);  
  
  pinMode(RELAY_PIN, OUTPUT);  
  digitalWrite(RELAY_PIN, LOW); // Solenoid OFF initially  
  
  // Connect to WiFi  
  Serial.println("Connecting to WiFi...");
```

```
WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("\nWiFi Connected!");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

server.begin();
}

void loop() {
    int sensorValue = analogRead(SENSOR_PIN);

    // Auto Control Based on Sensor
    if (sensorValue < threshold) {
        digitalWrite(RELAY_PIN, HIGH); // Turn ON solenoid
    } else {
        digitalWrite(RELAY_PIN, LOW); // Turn OFF solenoid
    }

    WiFiClient client = server.available();
    if (!client) return;

    String req = client.readStringUntil('\r');
    client.flush();

    // Manual control from webpage
    if (req.indexOf("/ON") != -1)
        digitalWrite(RELAY_PIN, HIGH);

    if (req.indexOf("/OFF") != -1)
        digitalWrite(RELAY_PIN, LOW);

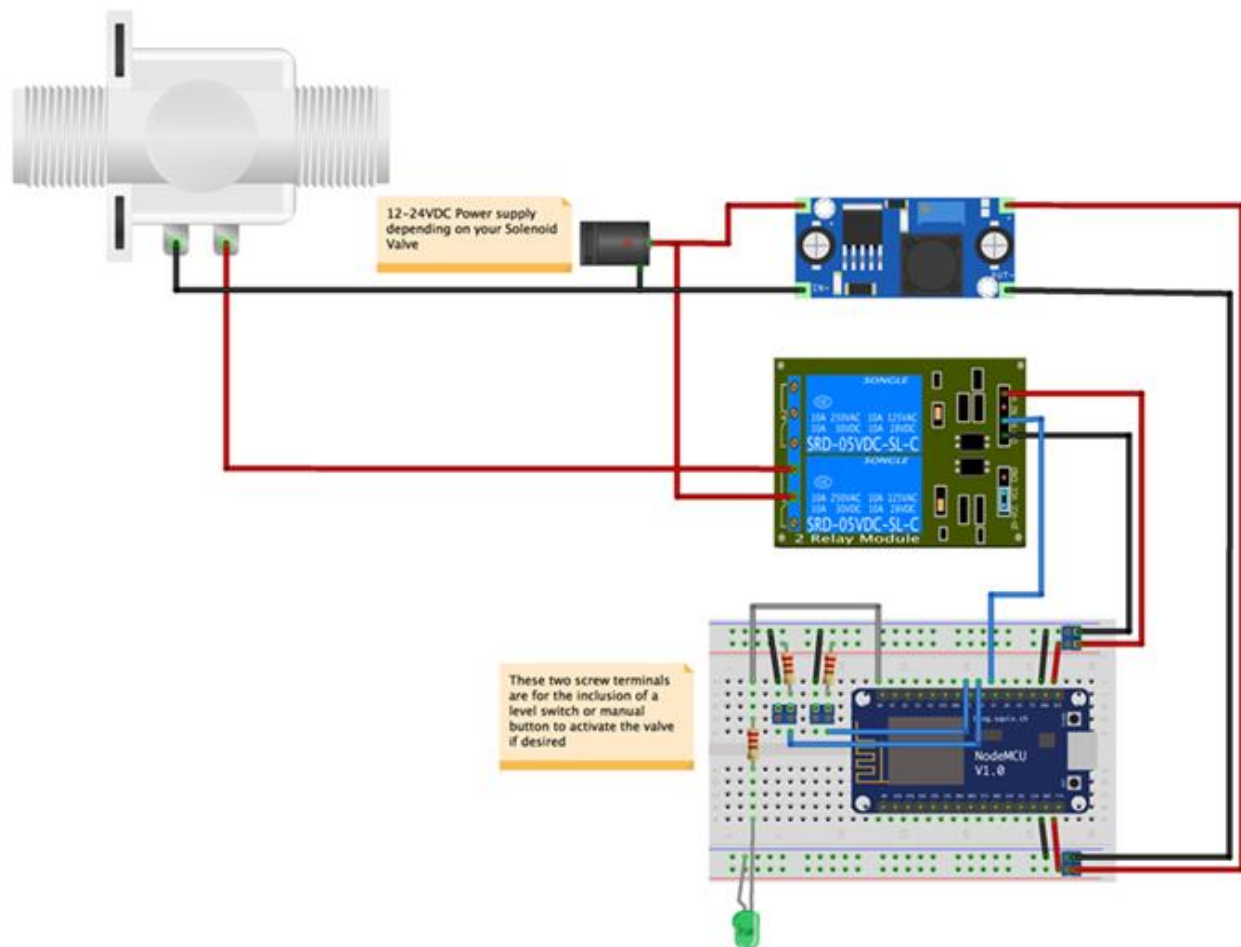
    // Web Page
    String html = "<html><body style='text-align:center;font-family:Arial'>";
    html += "<h1>IoT Sensor + Solenoid Valve Control</h1>";
    html += "<h2>Sensor Value: " + String(sensorValue) + "</h2>";
    html += "<a href='/ON'><button style='padding:10px 30px;'>Valve ON</button></a>";
    html += "<a href='/OFF'><button style='padding:10px 30px;'>Valve OFF</button></a>";
    html += "<p>Auto control also active based on sensor threshold.</p>";
    html += "</body></html>";
```

```

client.println("HTTP/1.1 200 OK");
client.println("Content-type:text/html");
client.println();
client.print(html);
client.stop();
}

```

Circuit Diagram of Hardware of Experiment:



RESULT:

Hence the sensor was successfully interfaced with the ESP32 microcontroller. The sensor data was displayed on a web page accessible through the Internet. The solenoid valve could be **turned ON/OFF remotely** using the webpage buttons. Additionally, the system automatically controlled the solenoid valve based on real-time sensor readings. This demonstrates a complete **IoT-based remote monitoring and control system**.

Experiment: 14	To interface sensor with microcontroller/SBC and write a program to turn ON/OFF Linear/Rotary Actuator through IoT when sensor data is detected.
Date:	

AIM:

To interface a sensor with a microcontroller or single-board computer (SBC) and implement an IoT-based system that **automatically turns ON/OFF a linear or rotary actuator when specific sensor data is detected** and also allows remote control through the Internet.

Apparatus Required:

SNO	Equipment Name	Quantity
1	Microcontroller / SBC – ESP32 / Arduino + NodeMCU / Raspberry Pi	1
2	Sensor (any suitable) – Ultrasonic sensor / Temperature sensor / IR sensor / Soil moisture sensor, etc.	1
3	Linear/Rotary Actuator – 12V Linear Actuator OR – DC Motor / Servo Motor (Rotary Actuator)	1
4	Motor Driver / Relay Module – L298N, L293D, or Relay for actuator control	1
5	Power Supply – 5V for controller, 12V for actuator	1
6	Connecting wires & Breadboard	1
7	Wi-Fi Internet connection	1
8	Laptop with Arduino IDE / Python (if using Raspberry Pi)	1

Software:

- Arduino IDE (for ESP32/Arduino)
- Thonny/Python (for Raspberry Pi)

Procedure:

1. Hardware Connections

Sensor → Microcontroller

- Connect sensor VCC → 3.3V/5V
- GND → GND
- Output pin → any analog/digital input pin

Actuator → Motor Driver / Relay → Microcontroller

- Actuator connected to motor driver/relay terminals
- Motor driver IN1/IN2 or relay IN → GPIO of controller
- External power supply connected to actuator

2. IoT Setup

- Configure ESP32/Raspberry Pi to connect to Wi-Fi
- Create a simple **Web Server** for remote ON/OFF control
- Display sensor values on webpage

3. Programming Steps

1. Initialize Wi-Fi connection
2. Read sensor values continuously
3. Compare sensor reading with a predefined threshold
4. If threshold is crossed → turn actuator ON
5. Else → turn actuator OFF
6. Provide buttons on web page for manual IoT control
7. Update actuator status in real time on webpage

4. Upload the Program

- Upload code using Arduino IDE (ESP32/Arduino)
- Or run Python script on Raspberry Pi

5. Testing

- ✓ Open Serial Monitor to see IP address
- ✓ Type IP address in browser
- ✓ View sensor data and actuator ON/OFF controls
- ✓ Change sensor environment to trigger actuator
- ✓ Confirm automatic + IoT-based control both work

Connections Used:

Ultrasonic Sensor → ESP32

- VCC → 5V
- GND → GND
- TRIG → GPIO 4
- ECHO → GPIO 15

Actuator → Relay Module → ESP32

- Relay IN → GPIO 5
- Relay VCC → 5V
- Relay GND → GND
- Actuator connected to relay NO + COM
- 12V supply for actuator

```
#include <WiFi.h>
```

```
const char* ssid = "YOUR_WIFI_NAME";  
const char* password = "YOUR_WIFI_PASSWORD";
```

```
// Sensor pins  
#define TRIG_PIN 4  
#define ECHO_PIN 15
```

```
// Actuator control pin (Relay/Motor Driver)  
#define ACTUATOR_PIN 5
```

```
long duration;  
int distance;  
int threshold = 20; // distance threshold in cm
```

```
WiFiServer server(80);
```

```
void setup() {  
  Serial.begin(115200);
```

```
  pinMode(TRIG_PIN, OUTPUT);  
  pinMode(ECHO_PIN, INPUT);
```

```
  pinMode(ACTUATOR_PIN, OUTPUT);  
  digitalWrite(ACTUATOR_PIN, LOW); // Actuator OFF initially
```

```
  // Connect WiFi  
  Serial.println("Connecting to WiFi...");  
  WiFi.begin(ssid, password);
```

```
  while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
  }
```

```
Serial.println("\nWiFi Connected!");
Serial.print("IP Address: ");

Serial.println(WiFi.localIP());

server.begin();
}

int readDistance() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    duration = pulseIn(ECHO_PIN, HIGH);
    int dist = duration * 0.034 / 2;
    return dist;
}

void loop() {

    // Read ultrasonic distance
    distance = readDistance();

    // Automatic actuator control
    if (distance < threshold) {
        digitalWrite(ACTUATOR_PIN, HIGH); // Actuator ON
    } else {
        digitalWrite(ACTUATOR_PIN, LOW); // Actuator OFF
    }

    WiFiClient client = server.available();
    if (!client) return;

    String request = client.readStringUntil('\r');
    client.flush();

    // Manual IoT control
    if (request.indexOf("/ON") != -1) {
        digitalWrite(ACTUATOR_PIN, HIGH);
    }
    if (request.indexOf("/OFF") != -1) {
        digitalWrite(ACTUATOR_PIN, LOW);
    }
}
```

```
// Web page interface
String html = "<html><body style='text-align:center; font-family:Arial'>";

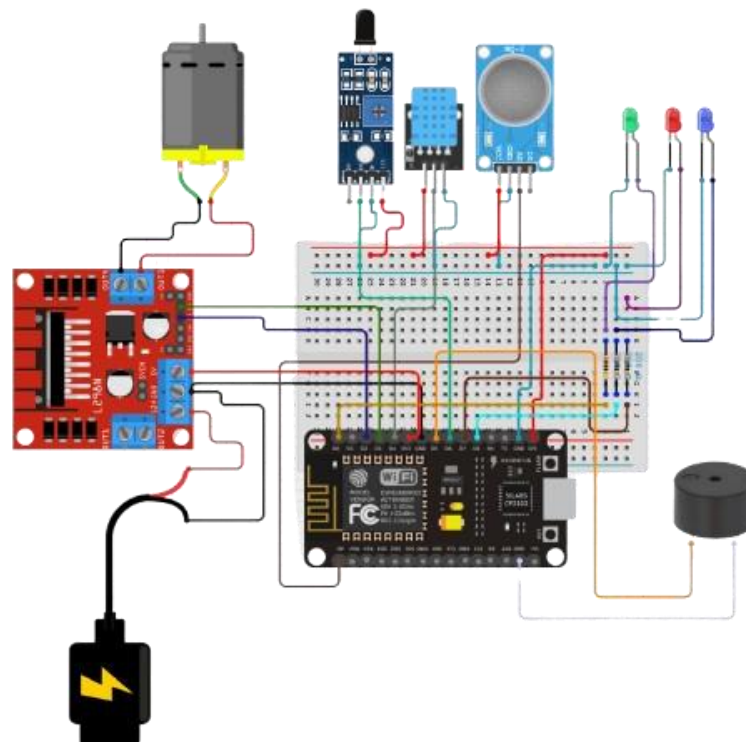
html += "<h1>IoT Sensor + Actuator Control</h1>";
html += "<h2>Distance: " + String(distance) + " cm</h2>";

html += "<a href='/ON'><button style='padding:10px 25px; font-size:20px;'>Actuator
ON</button></a>";
html += "<a href='/OFF'><button style='padding:10px 25px; font-size:20px;'>Actuator
OFF</button></a>";

html += "<p>Automatic Control Active: Actuator turns ON when distance < " + String(threshold)
+ " cm </p>";
html += "</body></html>";

client.println("HTTP/1.1 200 OK");
client.println("Content-type:text/html");
client.println();
client.print(html);
client.stop();
}
```

Hardware Circuit Diagram:



RESULT:

The sensor was successfully interfaced with the microcontroller/SBC, and the actuator was effectively controlled using IoT. The actuator (linear or rotary) automatically turned **ON/OFF** when the sensor data crossed the threshold. The system also allowed **remote operation through an IoT web interface**. This demonstrates a functional **IoT-based monitoring and control system**, suitable for automation, robotics, smart agriculture, and industrial applications.

Experiment: 15	To interface Bluetooth/Wifi with microcontroller/SBC and write a program to send sensor data to smart phone using Bluetooth/wifi.
Date:	

AIM:

To interface a sensor with a NodeMCU (ESP8266) microcontroller and write a program to send real-time sensor data to a smartphone using **Bluetooth (optional ESP32)** or **Wi-Fi (NodeMCU)** through a mobile web page.

Apparatus Required:

SNO	Equipment Name	Quantity
	Hardware	
1	NodeMCU ESP8266 (Wi-Fi-enabled microcontroller)	1
2	Sensor (Any analog or digital sensor –LM35 Temperature Sensor)	1
3	Smartphone	1
4	USB Cable	1
5	Jumper wires / Breadboard	1
6	Power supply (5V via USB)	1
	Software	
8	Arduino IDE	1
	Node MCU Board Package	1
	Wi-Fi hotspot or router	1

PROCEDURE:**1. Hardware Connections (Example: LM35 Sensor)**

- LM35 VCC → 3.3V
- LM35 GND → GND
- LM35 OUT → A0 pin of NodeMCU

2. Arduino IDE Setup

1. Open Arduino IDE
2. Install ESP8266 board:
File → Preferences → Add URL:
http://arduino.esp8266.com/stable/package_esp8266com_index.json
3. Go to:
Tools → Board → ESP8266 → NodeMCU 1.0
4. Select correct COM port

3. Program Logic

- Read analog sensor data
- Convert reading (e.g., to temperature)
- Create a **Wi-Fi web server**
- Send sensor data to smartphone on a webpage
- Smartphone accesses the data by entering NodeMCU IP address

4. Upload Program

- Connect NodeMCU → USB
- Verify & Upload

5. Testing

1. Open **Serial Monitor**
2. Note the **IP address** displayed
3. Open the IP address in your smartphone browser
4. Sensor data will appear live

CODE (NodeMCU ESP8266 – Sending Sensor Data to Smartphone via Wi-Fi):

Sensor: LM35 Temperature Sensor

```
#include <ESP8266WiFi.h>
```

```
// Enter Wi-Fi credentials
```

```
const char* ssid = "YOUR_WIFI_NAME";
```

```
const char* password = "YOUR_WIFI_PASSWORD";
```

```
WiFiServer server(80);
```

```
void setup() {  
  Serial.begin(115200);
```

```
  // Start Wi-Fi
```

```
  Serial.println("Connecting to WiFi...");
```

```
  WiFi.begin(ssid, password);
```

```
  while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
  }
```

```
  Serial.println("\nWiFi Connected!");
```

```
  Serial.print("IP Address: ");
```

```
  Serial.println(WiFi.localIP());
```

```
server.begin();
}

void loop() {
  WiFiClient client = server.available();
  if (!client) return;

  client.readStringUntil("\r"); // Read request
  client.flush();

  // Read sensor (LM35 connected to A0)
  int raw = analogRead(A0);
  float voltage = raw * (3.3 / 1023.0);
  float temperature = voltage * 100; // LM35: 10mV = 1°C

  // Web Page HTML
  String html = "<html><body style='text-align:center; font-family:Arial'>";
  html += "<h1>IoT Sensor Data (NodeMCU)</h1>";
  html += "<h2>Temperature: ";
  html += String(temperature);
  html += " °C</h2>";
  html += "<p>Refresh the page to update data.</p>";
  html += "</body></html>";

  // Send to smartphone
  client.println("HTTP/1.1 200 OK");
  client.println("Content-type:text/html");
  client.println();
  client.print(html);
  client.stop();
}
```

Using DHT11 and Blynk for Wi-Fi communication:

```
#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
#include <DHT.h>

// You should get Auth Token in the Blynk App.
// Go to the Project Settings (nut icon).
char auth[] = "YOUR_BLYNK_AUTH_TOKEN";
```

```
// Your WiFi credentials.
// Set password to "" for open networks.
char ssid[] = "YOUR_WIFI_SSID";
char pass[] = "YOUR_WIFI_PASSWORD";

#define DHTPIN D2      // DHT sensor connected to D2
#define DHTTYPE DHT11  // DHT 11

DHT dht(DHTPIN, DHTTYPE);

BlynkTimer timer;

void sendSensorData()
{
  float h = dht.readHumidity();
  float t = dht.readTemperature();

  if (isnan(h) || isnan(t)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

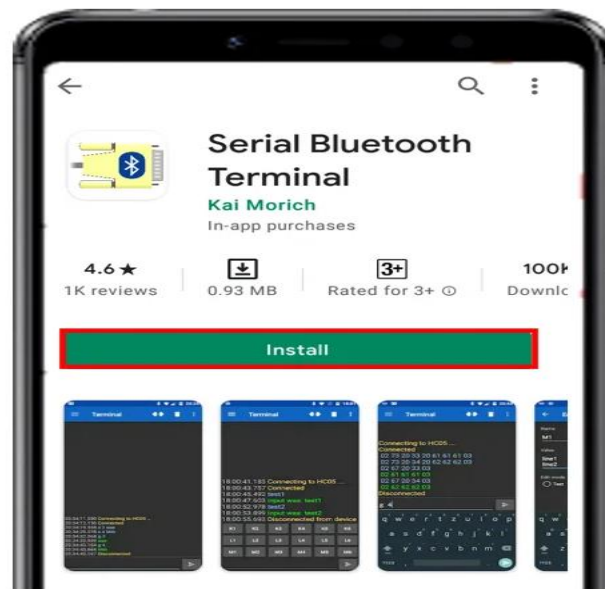
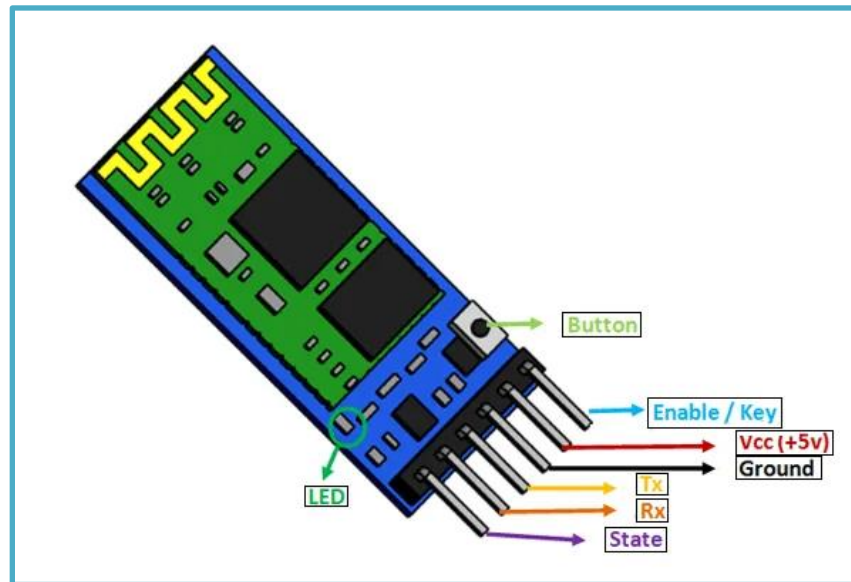
  Blynk.virtualWrite(V5, h); // Send humidity to Virtual Pin V5
  Blynk.virtualWrite(V6, t); // Send temperature to Virtual Pin V6
}

void setup()
{
  Serial.begin(115200);
  Blynk.begin(auth, ssid, pass);
  dht.begin();

  // Setup a function to be called every 2 seconds
  timer.setInterval(2000L, sendSensorData);
}

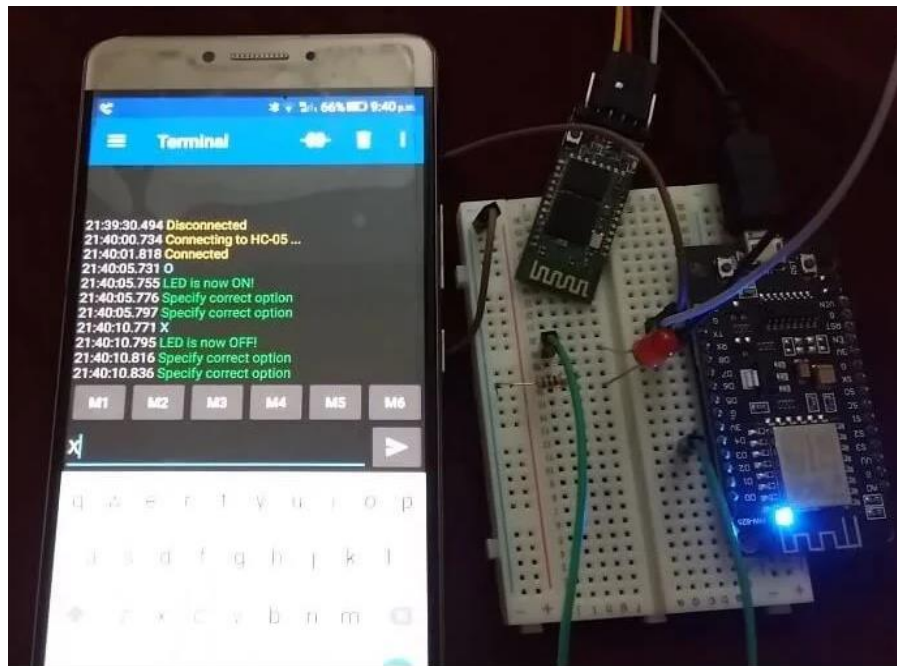
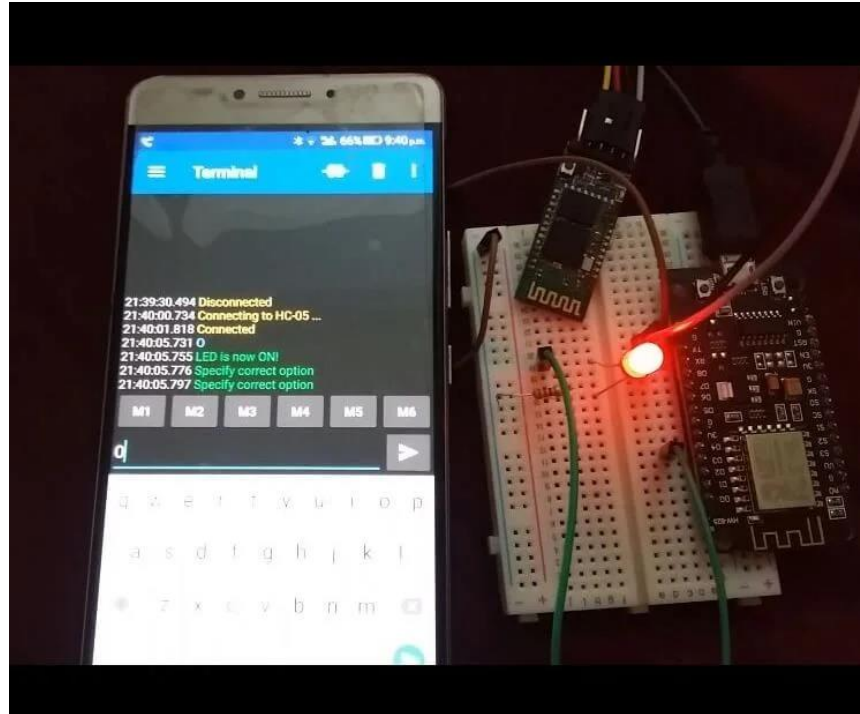
void loop()
{
  Blynk.run();
  timer.run();
}
```


Bluetooth Module pin out details:



Phone Device Bluetooth App connectivity app download

Circuit hardware output of Bluetooth Module and NodeMCU with Smartphone Communication:



RESULT:

Hence the NodeMCU successfully connected to the Wi-Fi network and sent sensor data wirelessly to the smartphone. The user was able to view real-time temperature sensor readings through a web page displayed on the mobile browser using the NodeMCU's IP address. This proves the successful implementation of an **IoT-based wireless sensor monitoring system**.

Lab In charge & Lab Manuel prepared By:

Dr. Chandrashekhar Kumar

AP/ECE

Verified By:

